OAVS • TGT COMPUTER SCIENCE TEACHER EXAMINATION

MCQ Question Bank

Complete Practice & Revision Set with Answers & Explanations

SUBJECT TOPIC

Loop Control Structure (C Programming)

256	14	100%
TOTAL MCQS	CHAPTERS	ANSWERS & EXPLANATIONS

Table of Contents

1. **Concept of Loop** (20 MCQs) 3

2. **Loops Supported by C** (15 MCQs) 9

3. **Top-Tested and Bottom-Tested Loops** (15 MCQs) 13

4. **The for Loop Statement** (26 MCQs) 18

5. **Nested for Loop** (15 MCQs) 25

6. **for Loop Variants** (20 MCQs) 30

7. **The while Loop Statement** (20 MCQs) 36

8. **Simple and Nested while Loop** (15 MCQs) 42

9. **Increment and Decrement Operators** (20 MCQs) 47

10. **Use of break** (15 MCQs) 53

11. **Use of continue** (15 MCQs) 58

12. **The do-while Loop** (20 MCQs) 63

13. **Comparison Among for, while, and do-while** (20 MCQs) 69

14. **Mixed Review and Exam-Pattern MCQs** (20 MCQs) 76

CHAPTER 1 Concept of Loop

20 Questions

Q1 **Definition** *Loop Control Structure • Concept of Loop*

What is a loop in the context of C programming?

A. A function that returns a value

B. A control structure that repeats a block of statements until a condition is met

C. A set of statements executed only once

D. A data type used to store integers

Ans: B. A control structure that repeats a block of statements until a condition is met

Explanation: A loop is a control structure that repeats a block of statements repeatedly as long as a specified condition holds true.

Q2 **Purpose** *Loop Control Structure • Concept of Loop*

What is the primary purpose of using loops in a program?

A. To allocate memory dynamically

B. To end program execution

C. To repeat a set of instructions without rewriting code

D. To declare variables

Ans: C. To repeat a set of instructions without rewriting code

Explanation: Loops allow a block of code to be executed multiple times without duplicating the statements, saving effort and code length.

Q3 **Advantage** *Loop Control Structure • Concept of Loop*

Which of the following is a major advantage of using loops?

A. Makes the program slower

B. Increases the size of source code unnecessarily

C. Reduces code redundancy and improves efficiency

D. Removes the need for variables

Ans: C. Reduces code redundancy and improves efficiency

Explanation: Loops reduce repetitive code, making programs shorter, more efficient, and easier to maintain.

Q4 **Conceptual** *Loop Control Structure • Concept of Loop*

Repetition of a set of statements a specific or unspecified number of times is called:

A. Iteration
B. Compilation
C. Recursion only
D. Branching

Ans: A. Iteration

Explanation: Iteration refers to the repeated execution of a block of statements, which is the fundamental concept behind loops.

Q5 **Conceptual** *Loop Control Structure • Concept of Loop*

Iterative processing in programming refers to:

A. Repeatedly executing a task until a termination condition is satisfied
B. Skipping statements randomly
C. Executing a task once and stopping
D. Executing tasks in reverse order

Ans: A. Repeatedly executing a task until a termination condition is satisfied

Explanation: Iterative processing means executing the same sequence of steps repeatedly until a defined condition causes termination.

Q6 **Application** *Loop Control Structure • Concept of Loop*

Loops are most useful in situations where:

A. A task needs to be performed repeatedly under a certain condition
B. A program has no variables
C. A program has no conditions
D. A task needs to be performed only once

Ans: A. A task needs to be performed repeatedly under a certain condition

Explanation: Loops are designed for tasks that must be repeated multiple times, such as processing lists of data or performing repeated calculations.

Q7 **Basic** *Loop Control Structure • Concept of Loop*

Which of the following best describes 'repetition' in loop terminology?

A. Executing a statement exactly one time
B. Calling a function recursively
C. Declaring the same variable twice
D. Executing the same block of code multiple times

Ans: D. Executing the same block of code multiple times

Explanation: Repetition means the same block of code is executed multiple times, which is the essence of looping.

Q8 **Application** *Loop Control Structure • Concept of Loop*

Which of the following tasks is best suited for implementation using a loop?

A. Adding two numbers once
B. Declaring a single variable
C. Printing numbers from 1 to 100
D. Printing 'Hello' once

Ans: C. Printing numbers from 1 to 100

Explanation: Printing a sequence of numbers repeatedly is a classic use case for loops, avoiding the need to write 100 separate print statements.

Q9 **Conceptual** *Loop Control Structure • Concept of Loop*

Without loops, repeating a task 100 times in a program would require:

A. Writing the statement only once
B. Writing the statement 100 times manually
C. Using only variables
D. Using no code at all

Ans: B. Writing the statement 100 times manually

Explanation: In the absence of loops, a programmer would have to duplicate the same statement manually for every repetition, which is inefficient.

Q10 **Definition** *Loop Control Structure • Concept of Loop*

A loop that continues to execute as long as a given condition remains true is controlled by:

A. A header file
B. An unconditional statement
C. A comment
D. A conditional test

Ans: D. A conditional test

Explanation: Loop execution is controlled by a conditional test that determines whether the loop body executes again or terminates.

Q11 **Advantage** *Loop Control Structure • Concept of Loop*

Which of the following is NOT an advantage of using loops?

A. Improves code readability for repetitive tasks
B. Increases manual repetition of code lines
C. Saves programmer's time
D. Reduces code length

Ans: B. Increases manual repetition of code lines

Explanation: Loops are meant to eliminate manual repetition of code, so 'increases manual repetition' is not an advantage but a drawback of not using loops.

Q12 **Conceptual** *Loop Control Structure • Concept of Loop*

The block of statements that is repeated in a loop is known as the:

A. Loop body
B. Loop pointer
C. Loop variable
D. Loop header

Ans: A. Loop body

Explanation: The set of statements that gets executed repeatedly inside a loop is referred to as the loop body.

Q13 **Application** *Loop Control Structure • Concept of Loop*

Which real-world programming task is a typical application of loops?

A. Defining a macro
B. Declaring a constant
C. Displaying a single welcome message
D. Summing all elements of a list of numbers

Ans: D. Summing all elements of a list of numbers

Explanation: Summing elements of a list requires repeated addition, which is efficiently handled using loops.

Q14 **Assertion-Reason** *Loop Control Structure • Concept of Loop*

Assertion (A): Loops help reduce the number of lines of code for repetitive tasks.
Reason (R): Loops execute a block of statements multiple times based on a condition.
Choose the correct option:

A. Both A and R are true, and R is the correct explanation of A
B. A is false, R is true
C. A is true, R is false
D. Both A and R are true, but R is not the correct explanation of A

Ans: A. Both A and R are true, and R is the correct explanation of A

Explanation: Loops reduce code lines precisely because they repeat a block based on a condition, so R correctly explains A.

Q15 **Tricky** *Loop Control Structure • Concept of Loop*

If a task needs to be performed exactly once with no repetition, which control structure is most appropriate?

A. Nested loop
B. Loop
C. Infinite loop
D. Simple sequential statement

Ans: D. Simple sequential statement

Explanation: A simple sequential (non-looping) statement is appropriate when a task is to be performed only once, not a loop.

Q16 **Conceptual** *Loop Control Structure • Concept of Loop*

Which term describes a loop that never terminates due to an always-true condition?

A. Finite loop
B. Infinite loop
C. Broken loop
D. Empty loop

Ans: B. Infinite loop

Explanation: An infinite loop is one whose controlling condition never becomes false, so it keeps executing indefinitely.

Q17 **Fill-in-the-Blank** *Loop Control Structure • Concept of Loop*

A loop is also referred to as a/an _____ in programming terminology.

A. Selection structure
B. Iterative structure
C. Sequence structure
D. Storage structure

Ans: B. Iterative structure

Explanation: Loops are classified as iterative structures because they repeat execution of statements.

Q18 **Basic** *Loop Control Structure • Concept of Loop*

Which of the following is NOT a benefit of iterative processing?

A. Reduced chance of manual copy errors
B. Efficient handling of repetitive tasks
C. Shorter and cleaner source code
D. Automatic elimination of all program bugs

Ans: D. Automatic elimination of all program bugs

Explanation: Loops do not automatically eliminate bugs; they only help avoid repetitive code, so this option is false and hence the correct answer for what is NOT a benefit.

Q19 **Scenario** *Loop Control Structure • Concept of Loop*

A programmer needs to print the multiplication table of a number from 1 to 10. Which construct should be used?

A. A structure definition
B. A loop
C. A single print statement
D. A pointer declaration

Ans: B. A loop

Explanation: Printing 10 repeated calculations (multiplication table entries) is a classic loop application.

Q20 **Conceptual** *Loop Control Structure • Concept of Loop*

In terms of control flow, a loop repeatedly transfers control back to:

A. The main function only

B. The beginning of the loop body until the condition fails

C. The end of the program

D. A random statement

Ans: B. The beginning of the loop body until the condition fails

Explanation: A loop repeatedly transfers control back to the start of the loop body as long as the condition remains true, and exits when it becomes false.

CHAPTER 2 Loops Supported by C

15 Questions

Q21 **Basic** *Loop Control Structure • Loops Supported by C*

How many types of loops are supported by the C programming language?

A. Two
B. Five
C. Three
D. Four

Ans: C. Three

Explanation: C supports three types of loops: for, while, and do-while.

Q22 **Basic** *Loop Control Structure • Loops Supported by C*

Which of the following is NOT a loop supported by C?

A. do-while
B. while
C. repeat-until
D. for

Ans: C. repeat-until

Explanation: C does not support a 'repeat-until' loop; that construct exists in languages like Pascal, not in standard C.

Q23 **Conceptual** *Loop Control Structure • Loops Supported by C*

Which loop in C is generally preferred when the number of iterations is known in advance?

A. while loop
B. for loop
C. None of these
D. do-while loop

Ans: B. for loop

Explanation: The for loop is typically used when the number of iterations is known beforehand, since it combines initialization, condition, and update in one line.

Q24 **Conceptual** *Loop Control Structure • Loops Supported by C*

Which loop in C is generally preferred when the number of iterations is NOT known in advance?

A. while loop
B. for loop
C. None of these
D. Both while and do-while are suitable

Ans: D. Both while and do-while are suitable

Explanation: When the number of iterations is unknown beforehand, while and do-while loops are typically used since they depend on a condition rather than a counter.

Q25 **Characteristic** *Loop Control Structure • Loops Supported by C*

Which characteristic is common to all three loops (for, while, do-while) in C?

A. They all must be nested
B. They all execute the body at least once
C. They all use the same keyword
D. They all require a controlling condition

Ans: D. They all require a controlling condition

Explanation: All three loop types in C rely on a controlling (test) condition to determine whether to continue or stop iterating.

Q26 **Difference** *Loop Control Structure • Loops Supported by C*

What is a key difference between the while loop and the do-while loop in C?

A. Both check the condition at the same time
B. while checks the condition before execution; do-while checks it after
C. Neither loop uses a condition
D. while checks the condition after execution; do-while checks it before

Ans: B. while checks the condition before execution; do-while checks it after

Explanation: The while loop tests its condition before executing the body (top-tested), whereas the do-while loop tests its condition after executing the body (bottom-tested).

Q27 **Difference** *Loop Control Structure • Loops Supported by C*

Which loop structure in C combines initialization, condition, and increment/decrement in a single statement?

A. for loop
B. do-while loop
C. None of these
D. while loop

Ans: A. for loop

Explanation: The for loop uniquely combines initialization, condition testing, and increment/decrement into one compact statement line.

Q28 **Application** *Loop Control Structure • Loops Supported by C*

Which loop would be most suitable for reading user input until the user enters a sentinel value, assuming at least one read must always occur?

A. for loop
B. No loop needed
C. goto statement
D. do-while loop

Ans: D. do-while loop

Explanation: A do-while loop is well suited here because it guarantees at least one execution, which fits scenarios requiring at least one input read before checking a condition.

Q29 **Tricky** *Loop Control Structure • Loops Supported by C*

Which of the following statements is true regarding loops in C?

A. All three loop types (for, while, do-while) can generally be used interchangeably with proper adjustments

B. do-while is a top-tested loop

C. Any task done using a for loop cannot be done using a while loop

D. for loop cannot contain an increment expression

Ans: A. All three loop types (for, while, do-while) can generally be used interchangeably with proper adjustments

Explanation: In C, for, while, and do-while loops are functionally interchangeable in most cases with proper adjustment of initialization, condition, and update expressions.

Q30 **Conceptual** *Loop Control Structure • Loops Supported by C*

The three loop constructs in C are classified based on:

A. When the controlling condition is tested and other structural differences

B. The number of variables they use

C. The color of syntax highlighting

D. The compiler version used

Ans: A. When the controlling condition is tested and other structural differences

Explanation: Loops in C are primarily classified based on when their condition is tested (before or after the loop body) and their syntactic structure.

Q31 **Match-the-Following** *Loop Control Structure • Loops Supported by C*

Match the loop type with its testing behavior:
1. for loop A. Tests condition after execution
2. while loop B. Tests condition before execution
3. do-while loop C. Tests condition before execution
Choose the correct match:

A. 1-B, 2-B, 3-A

B. 1-A, 2-B, 3-C

C. 1-A, 2-A, 3-B

D. 1-C, 2-A, 3-B

Ans: A. 1-B, 2-B, 3-A

Explanation: Both for and while loops test their condition before execution (top-tested), while do-while tests its condition after execution (bottom-tested).

Q32 **Basic** *Loop Control Structure • Loops Supported by C*

Which of these loop keywords are reserved words in C? (Select the correct group)

A. loop, while, do
B. for, while, repeat
C. for, until, repeat
D. for, while, do

Ans: D. for, while, do

Explanation: The reserved keywords for looping in C are exactly: for, while, and do (used with do-while).

Q33 **Conceptual** *Loop Control Structure • Loops Supported by C*

In C, which loop is typically chosen for menu-driven programs that must display the menu at least once?

A. while loop
B. do-while loop
C. None, menus don't use loops
D. for loop

Ans: B. do-while loop

Explanation: The do-while loop is ideal for menu-driven programs because it guarantees the menu is displayed at least once before checking whether to repeat.

Q34 **Output-based** *Loop Control Structure • Loops Supported by C*

Which loop structure would execute zero times if its initial condition is false from the start?

A. Only for loop
B. do-while loop
C. None; all loops execute at least once
D. for loop and while loop

Ans: D. for loop and while loop

Explanation: Both for and while loops test their condition before the first execution, so if the condition is false initially, the loop body never executes.

Q35 **Previous-Year-Pattern** *Loop Control Structure • Loops Supported by C*

Which of the following is TRUE about the three loop types supported by C?

A. Only for loop can use break and continue
B. They all serve the same purpose of repeating a block of statements but differ in structure and condition testing
C. They all have identical syntax
D. do-while cannot use variables

Ans: B. They all serve the same purpose of repeating a block of statements but differ in structure and condition testing

Explanation: All three loops achieve repetition, but they differ in syntax and in when the condition is tested, giving flexibility for different scenarios.

CHAPTER 3 Top-Tested and Bottom-Tested Loops

15 Questions

Q36 **Definition** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

A top-tested loop is one in which the condition is checked:

A. Never checked
B. Before the loop body executes
C. After the loop body executes
D. Only once at compile time

Ans: B. Before the loop body executes

Explanation: In a top-tested loop, the condition is evaluated before the loop body executes, so the body may not execute at all if the condition is false.

Q37 **Definition** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

A bottom-tested loop is one in which the condition is checked:

A. Never
B. Only during compilation
C. Before the loop body executes
D. After the loop body executes

Ans: D. After the loop body executes

Explanation: In a bottom-tested loop, the condition is evaluated after the loop body executes, guaranteeing at least one execution.

Q38 **Example** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which of the following is an example of a top-tested loop in C?

A. while loop
B. switch statement
C. do-while loop
D. goto statement

Ans: A. while loop

Explanation: The while loop checks its condition before executing the body, making it a top-tested loop.

Q39 **Example** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which of the following is an example of a bottom-tested loop in C?

A. if-else statement
B. do-while loop
C. for loop
D. while loop

Ans: B. do-while loop

Explanation: The do-while loop checks its condition after executing the body, making it the bottom-tested loop in C.

Q40 **Characteristic** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which of the following is a defining characteristic of a bottom-tested loop?

A. It may never execute its body
B. It cannot use variables
C. It cannot be nested
D. It always executes its body at least once

Ans: D. It always executes its body at least once

Explanation: Because the condition is checked after execution, a bottom-tested loop guarantees the body runs at least one time.

Q41 **Characteristic** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which of the following is a defining characteristic of a top-tested loop?

A. It always executes at least once regardless of the condition
B. It always runs infinitely
C. It may execute zero times if the condition is initially false
D. It cannot be terminated using break

Ans: C. It may execute zero times if the condition is initially false

Explanation: A top-tested loop checks the condition first, so if it is false at the outset, the loop body is skipped entirely, resulting in zero executions.

Q42 **Comparison** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which statement correctly compares top-tested and bottom-tested loops?

A. Bottom-tested loops guarantee at least one execution, while top-tested loops do not
B. Both guarantee the same number of executions always
C. Bottom-tested loops cannot use conditions
D. Top-tested loops always execute more times than bottom-tested loops

Ans: A. Bottom-tested loops guarantee at least one execution, while top-tested loops do not

Explanation: The key distinguishing feature is that bottom-tested loops always execute the body once before testing, unlike top-tested loops.

Q43 **Advantage** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

What is an advantage of a bottom-tested loop like do-while in scenarios such as input validation?

A. It skips input reading entirely

B. It ensures the input is read and processed at least once before validation

C. It prevents any user input

D. It cannot check conditions

Ans: B. It ensures the input is read and processed at least once before validation

Explanation: Since do-while executes the body first, it is advantageous for input validation where at least one prompt/read must occur before checking correctness.

Q44 **Conceptual** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

For and while loops in C are classified as:

A. Bottom-tested loops

B. Top-tested loops

C. Infinite loops only

D. Neither top nor bottom tested

Ans: B. Top-tested loops

Explanation: Both for and while loops evaluate their condition before executing the loop body, so both are top-tested loops.

Q45 **Assertion-Reason** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Assertion (A): The do-while loop is called a bottom-tested loop.
Reason (R): Its condition is evaluated after the loop body executes at least once.
Choose the correct option:

A. A is true, R is false

B. Both A and R are true, and R correctly explains A

C. Both A and R are true, but R does not explain A

D. A is false, R is true

Ans: B. Both A and R are true, and R correctly explains A

Explanation: The do-while loop is indeed bottom-tested precisely because its condition check occurs after the loop body's first execution, so R correctly explains A.

Q46 **Tricky** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

If the condition in a do-while loop is false right from the beginning, how many times will the loop body execute?

A. Infinite times
B. Exactly one time
C. Zero times
D. Cannot be determined

Ans: B. Exactly one time

Explanation: Since do-while checks the condition only after executing the body, the body always executes at least once, even if the condition is false from the start.

Q47 **Tricky** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

If the condition in a while loop is false right from the beginning, how many times will the loop body execute?

A. At least twice
B. Zero times
C. Exactly one time
D. Infinite times

Ans: B. Zero times

Explanation: Since a while loop checks its condition before executing, a false condition at the start results in the loop body never executing.

Q48 **Scenario** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

A program must display a menu at least once even if the user immediately chooses to exit. Which type of loop (top-tested or bottom-tested) fits this best?

A. Only nested loops work
B. Top-tested loop
C. Bottom-tested loop
D. Neither type works

Ans: C. Bottom-tested loop

Explanation: A bottom-tested loop (do-while) guarantees the menu displays at least once before the exit condition is checked, matching the requirement exactly.

Q49 **Comparison** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which of the following is a limitation of bottom-tested loops compared to top-tested loops?

A. They cannot contain any statements
B. They can never terminate
C. The loop body always executes at least once even when it may not be desired
D. They always run faster

Ans: C. The loop body always executes at least once even when it may not be desired

Explanation: A limitation of bottom-tested loops is that the body executes at least once even if the condition was never intended to be true, which can be undesirable in some scenarios.

Q50 **Conceptual** *Loop Control Structure • Top-Tested and Bottom-Tested Loops*

Which term best describes the moment when a loop's controlling condition is evaluated relative to its body, forming the basis for classifying it as top-tested or bottom-tested?

A. Execution order of the condition check versus the loop body

B. The data type of the loop counter

C. The name of the loop keyword

D. The number of variables declared

Ans: A. Execution order of the condition check versus the loop body

Explanation: The classification into top-tested or bottom-tested is based solely on whether the condition check happens before (top) or after (bottom) the loop body executes.

CHAPTER 4 The for Loop Statement

26 Questions

Q51 **Syntax** *Loop Control Structure • The for Loop Statement*

What is the correct general syntax of a for loop in C?

A. for[initialization; condition; increment]
B. for(increment; condition; initialization)
C. for(condition; initialization; increment)
D. for(initialization; condition; increment/decrement)

Ans: D. for(initialization; condition; increment/decrement)

Explanation: The standard for loop syntax in C is for(initialization; condition; increment/decrement) { statements; }.

Q52 **Components** *Loop Control Structure • The for Loop Statement*

How many parts (expressions) does the header of a for loop contain, separated by semicolons?

A. Three
B. Two
C. One
D. Four

Ans: A. Three

Explanation: The for loop header contains three expressions: initialization, condition, and increment/decrement, separated by two semicolons.

Q53 **Components** *Loop Control Structure • The for Loop Statement*

Which part of a for loop is executed only once, at the very beginning?

A. Increment/decrement
B. Condition
C. Initialization
D. Loop body

Ans: C. Initialization

Explanation: The initialization expression in a for loop executes only once, before the loop begins iterating.

Q54 **Execution-Flow** *Loop Control Structure • The for Loop Statement*

In a for loop, in what order are the three expressions logically evaluated during the first iteration?

A. Condition, Initialization, Body
B. Initialization, Condition, Body
C. Increment, Condition, Initialization
D. Body, Initialization, Condition

Ans: B. Initialization, Condition, Body

Explanation: The for loop first performs initialization, then checks the condition, and if true, executes the loop body.

Q55 **Execution-Flow** *Loop Control Structure • The for Loop Statement*

After the loop body executes in a for loop, which expression is evaluated next before re-checking the condition?

A. Condition
B. None, the loop ends
C. Initialization
D. Increment/decrement

Ans: D. Increment/decrement

Explanation: After the body executes, the increment/decrement expression is evaluated, and then the condition is re-checked to decide on the next iteration.

Q56 **Condition-Checking** *Loop Control Structure • The for Loop Statement*

When does the for loop terminate?

A. When the increment expression becomes zero
B. When the loop body is empty
C. When the initialization expression becomes true
D. When the condition expression evaluates to false

Ans: D. When the condition expression evaluates to false

Explanation: A for loop continues executing as long as its condition is true and terminates as soon as the condition evaluates to false.

Q57 **Code-Snippet** *Loop Control Structure • The for Loop Statement*

What will be the output of: for(int i=1; i<=3; i++) printf("%d ", i);

A. 3 2 1
B. 1 2 3 4
C. 0 1 2
D. 1 2 3

Ans: D. 1 2 3

Explanation: The loop starts at 1, continues while i<=3, and increments i by 1 each time, printing 1, 2, and 3.

Q58 **Output-based** *Loop Control Structure • The for Loop Statement*

What is the output of: for(int i=5; i>=1; i--) printf("%d ", i);

A. 1 to 5 in random order
B. 5 4 3 2 1 0
C. 5 4 3 2 1
D. 1 2 3 4 5

Ans: C. 5 4 3 2 1

Explanation: The loop starts at 5 and decrements down to 1, printing values in descending order: 5 4 3 2 1.

Q59 **Iteration-Count** *Loop Control Structure • The for Loop Statement*

How many times will the loop 'for(int i=0; i<10; i++)' execute?

A. 11 times
B. 10 times
C. 9 times
D. Infinite times

Ans: B. 10 times

Explanation: The loop runs for i = 0,1,2,...,9, which is exactly 10 iterations, since i<10 becomes false when i equals 10.

Q60 **Tricky** *Loop Control Structure • The for Loop Statement*

How many times will the loop 'for(int i=1; i<=10; i++)' execute?

A. 10 times
B. Infinite times
C. 11 times
D. 9 times

Ans: A. 10 times

Explanation: The loop runs for i = 1,2,...,10, giving exactly 10 iterations before i<=10 becomes false at i=11.

Q61 **Application** *Loop Control Structure • The for Loop Statement*

Which of the following tasks is a typical application of the for loop?

A. Defining a macro constant
B. Waiting indefinitely for a random event
C. Declaring a structure
D. Summing the first N natural numbers where N is known

Ans: D. Summing the first N natural numbers where N is known

Explanation: When the number of iterations (N) is known in advance, the for loop is the most natural and commonly used choice.

Q62 **Syntax** *Loop Control Structure • The for Loop Statement*

Which of the following is a syntactically valid for loop header in C?

A. for(i=0, i<5, i++)
B. for(i=0; i<5; i++)
C. for i=0; i<5; i++
D. for(i=0: i<5: i++)

Ans: B. for(i=0; i<5; i++)

Explanation: The correct syntax requires semicolons to separate the three expressions and parentheses to enclose them: for(i=0; i<5; i++).

Q63 **Conceptual** *Loop Control Structure • The for Loop Statement*

Which of the following is TRUE about the increment/decrement part of a for loop?

A. It must always use the ++ operator

B. It can use any valid expression that updates the loop control variable

C. It is mandatory and cannot be omitted

D. It cannot modify the loop variable

Ans: B. It can use any valid expression that updates the loop control variable

Explanation: The increment/decrement expression can be any valid C expression that updates the loop control variable, not restricted to ++ or -- only.

Q64 **Code-Snippet** *Loop Control Structure • The for Loop Statement*

What will be the output of: for(int i=0; i<5; i+=2) printf("%d ", i);

A. 0 2 4 6

B. 0 1 2 3 4

C. 0 2 4

D. 1 3 5

Ans: C. 0 2 4

Explanation: Since i increases by 2 each time starting from 0, the values printed are 0, 2, and 4 (loop stops when i becomes 6, since 6<5 is false).

Q65 **Output-based** *Loop Control Structure • The for Loop Statement*

What is the output of the following code?

```
int sum=0;
for(int i=1; i<=5; i++) sum += i;
printf("%d", sum);
```

A. 10

B. 20

C. 5

D. 15

Ans: D. 15

Explanation: The loop sums 1+2+3+4+5 = 15, so the output is 15.

Q66 **Scenario** *Loop Control Structure • The for Loop Statement*

A programmer wants to print 'Hello' exactly 7 times. Which for loop header is most appropriate?

A. for(int i=7; i<7; i++)

B. for(int i=0; i>7; i++)

C. for(int i=0; i<7; i--)

D. for(int i=0; i<7; i++)

Ans: D. for(int i=0; i<7; i++)

Explanation: for(int i=0; i<7; i++) correctly iterates 7 times (i=0 through i=6), printing 'Hello' 7 times.

Q67 **Tricky** *Loop Control Structure • The for Loop Statement*

What happens if the loop body of a for loop contains no statements (an empty body, e.g. for(i=0;i<5;i++);)?

A. The loop simply executes the empty statement repeatedly without any visible action, then terminates normally

B. The loop becomes infinite

C. Compilation error occurs

D. The program crashes

Ans: A. The loop simply executes the empty statement repeatedly without any visible action, then terminates normally

Explanation: An empty for loop body (with a semicolon) is syntactically valid; it just iterates without performing any visible operation and terminates when the condition becomes false.

Q68 **Conceptual** *Loop Control Structure • The for Loop Statement*

Can the initialization part of a for loop declare more than one variable using a comma operator?

A. Only using pointers

B. No, only one variable can be initialized

C. Only in while loops

D. Yes, multiple variables can be initialized using commas, e.g., for(i=0,j=10; ...)

Ans: D. Yes, multiple variables can be initialized using commas, e.g., for(i=0,j=10; ...)

Explanation: C allows multiple initializations in a for loop separated by commas, such as for(i=0, j=10; i<j; i++, j--).

Q69 **Application** *Loop Control Structure • The for Loop Statement*

Which of the following is a common application of the for loop in array processing?

A. Traversing all elements of an array using an index variable

B. Declaring the array only

C. Terminating the program early

D. Compiling the program

Ans: A. Traversing all elements of an array using an index variable

Explanation: The for loop is commonly used to traverse arrays because the number of elements (array size) is typically known in advance.

Q70 **Match-the-Following** *Loop Control Structure • The for Loop Statement*

Match the for loop component with its function:
1. Initialization A. Updates loop variable
2. Condition B. Sets starting value
3. Increment/Decrement C. Tests continuation
Correct match:

A. 1-B, 2-A, 3-C
B. 1-A, 2-B, 3-C
C. 1-C, 2-A, 3-B
D. 1-B, 2-C, 3-A

Ans: D. 1-B, 2-C, 3-A

Explanation: Initialization sets the starting value (B), condition tests whether to continue (C), and increment/decrement updates the loop variable (A).

Q71 **Previous-Year-Pattern** *Loop Control Structure • The for Loop Statement*

In the statement for(int i=1; i<=n; i++), what does 'i<=n' represent?

A. Increment expression
B. Initialization expression
C. Loop body
D. Condition expression

Ans: D. Condition expression

Explanation: 'i<=n' is the condition expression that is checked before every iteration to decide whether the loop should continue.

Q72 **Tricky** *Loop Control Structure • The for Loop Statement*

What is the output of: for(int i=10; i>10; i++) printf("%d", i);

A. Nothing is printed (loop body never executes)
B. Compilation error
C. 10
D. Infinite loop

Ans: A. Nothing is printed (loop body never executes)

Explanation: Since the condition i>10 is false right from the start (i=10 is not greater than 10), the loop body never executes and nothing is printed.

Q73 **Difficult** *Loop Control Structure • The for Loop Statement*

What is the output of: int i; for(i=0; i<5; i++){ if(i==3) i=10; } printf("%d", i);

A. 3
B. 10
C. 5
D. 11

Ans: D. 11

Explanation: When i becomes 3, it is reassigned to 10 inside the body; after the increment i becomes 11, and since 11<5 is false, the loop ends with i=11.

Q74 **Code-Snippet** *Loop Control Structure • The for Loop Statement*

What is the output of: for(int i=1; i<=3; i++){ for(int j=1; j<=2; j++){ printf("%d%d ", i, j); } }

A. 12 22 32
B. 11 12 13
C. 11 12 21 22 31 32
D. 11 21 31 12 22 32

Ans: C. 11 12 21 22 31 32

Explanation: For each value of i (1 to 3), the inner loop runs j from 1 to 2, producing pairs 11 12, then 21 22, then 31 32 in that order.

Q75 **Assertion-Reason** *Loop Control Structure • The for Loop Statement*

Assertion (A): The for loop is preferred when the number of iterations is known beforehand.
Reason (R): The for loop's header allows initialization, condition, and increment to be specified together, making iteration count control easy.
Choose the correct option:

A. Both A and R are true, but R does not explain A
B. A is true, R is false
C. A is false, R is true
D. Both A and R are true, and R correctly explains A

Ans: D. Both A and R are true, and R correctly explains A

Explanation: Because all three control expressions are grouped together, the for loop makes it easy to manage a known number of iterations, correctly explaining why it's preferred in such cases.

Q76 **Difficult** *Loop Control Structure • The for Loop Statement*

What is the output of: for(int i=0, j=5; i<j; i++, j--) printf("%d-%d ", i, j);

A. No output
B. 0-5 1-4 2-3 3-2
C. 0-5 1-4
D. 0-5 1-4 2-3

Ans: D. 0-5 1-4 2-3

Explanation: The loop runs while i<j: (0,5),(1,4),(2,3); at i=3,j=2 the condition i<j (3<2) is false, so the loop stops after printing three pairs.

CHAPTER 5 Nested for Loop

15 Questions

Q77 **Definition** *Loop Control Structure • Nested for Loop*

What is a nested for loop?

A. A for loop without an increment expression

B. A for loop placed inside the body of another for loop

C. A for loop that runs only once

D. A for loop that has no condition

Ans: B. A for loop placed inside the body of another for loop

Explanation: A nested for loop occurs when one for loop (the inner loop) is written inside the body of another for loop (the outer loop).

Q78 **Conceptual** *Loop Control Structure • Nested for Loop*

In a nested for loop, which loop completes all its iterations first for each single iteration of the outer loop?

A. The inner loop

B. Both complete simultaneously

C. Neither completes

D. The outer loop

Ans: A. The inner loop

Explanation: For every single iteration of the outer loop, the inner loop runs through all of its iterations completely before control returns to the outer loop.

Q79 **Execution-Order** *Loop Control Structure • Nested for Loop*

If the outer loop runs 3 times and the inner loop runs 4 times for each outer iteration, how many times does the inner loop body execute in total?

A. 12

B. 3

C. 7

D. 4

Ans: A. 12

Explanation: The inner loop body executes outer_iterations × inner_iterations = 3 × 4 = 12 times in total.

Q80 **Iteration-Analysis** *Loop Control Structure • Nested for Loop*

For the code: for(i=1;i<=2;i++){ for(j=1;j<=3;j++){ printf("*"); } }, how many asterisks are printed?

A. 2
B. 3
C. 6
D. 5

Ans: C. 6

Explanation: The outer loop runs 2 times and the inner loop runs 3 times for each, so total asterisks = 2 × 3 = 6.

Q81 **Application** *Loop Control Structure • Nested for Loop*

Nested for loops are commonly used for which of the following tasks?

A. Terminating a program
B. Printing patterns such as triangles made of stars
C. Reading one input value
D. Declaring a single variable

Ans: B. Printing patterns such as triangles made of stars

Explanation: Nested loops are widely used to print two-dimensional patterns, such as star triangles or number pyramids, since they require row-and-column control.

Q82 **Code-Snippet** *Loop Control Structure • Nested for Loop*

What does the following code print?

```
for(int i=1;i<=3;i++){
for(int j=1;j<=i;j++){
printf("*");
}
printf("\n");
}
```

A. A single row of 3 stars
B. A square of stars
C. No output
D. A right triangle pattern: *, **, ***

Ans: D. A right triangle pattern: *, **, *

Explanation: Since the inner loop runs from 1 to i, row 1 prints 1 star, row 2 prints 2 stars, and row 3 prints 3 stars, forming a right-triangle pattern.

Q83 **Conceptual** *Loop Control Structure • Nested for Loop*

In a nested loop, when the inner loop's condition becomes false, what happens next?

A. Control returns to the outer loop, which proceeds to its own increment and condition check

B. The entire program terminates

C. The inner loop restarts from zero

D. The outer loop is also terminated immediately

Ans: A. Control returns to the outer loop, which proceeds to its own increment and condition check

Explanation: When the inner loop's condition becomes false, only the inner loop ends; control returns to the outer loop to continue its own iteration cycle.

Q84 **Iteration-Analysis** *Loop Control Structure • Nested for Loop*

How many total iterations occur for: for(i=0;i<4;i++){ for(j=0;j<4;j++){ ... } }?

A. 20

B. 8

C. 4

D. 16

Ans: D. 16

Explanation: Both loops run 4 times each, so the inner block executes 4 × 4 = 16 times in total.

Q85 **Tricky** *Loop Control Structure • Nested for Loop*

Can the inner loop of a nested for loop use the same loop variable name as the outer loop (e.g., both named i) if declared in separate scopes?

A. Only if both loops are outside a function

B. No, this always causes a compilation error

C. Only in while loops

D. Yes, if each loop variable is declared within its own loop's scope (e.g., using for(int i...)), they are treated as separate variables

Ans: D. Yes, if each loop variable is declared within its own loop's scope (e.g., using for(int i...)), they are treated as separate variables

Explanation: If each loop declares its own variable within its own block scope, C allows reuse of the same name since they exist in different scopes without conflict.

Q86 **Difficult** *Loop Control Structure • Nested for Loop*

What is the output of: for(int i=1;i<=2;i++){ for(int j=1;j<=2;j++){ if(j==2) break; printf("%d%d ", i, j); } }

A. 11 21
B. 11 12 21 22
C. 11 21 12 22
D. No output

Ans: A. 11 21

Explanation: The break exits only the inner loop when j==2, so for each outer iteration only j=1 is printed, giving 11 and then 21.

Q87 **Application** *Loop Control Structure • Nested for Loop*

Nested for loops are essential for processing which type of data structure efficiently?

A. Boolean flags
B. Two-dimensional arrays (matrices)
C. Character constants
D. Single integer variables

Ans: B. Two-dimensional arrays (matrices)

Explanation: Two-dimensional arrays require an outer loop for rows and an inner loop for columns, making nested for loops essential for their traversal.

Q88 **Scenario** *Loop Control Structure • Nested for Loop*

To print a multiplication table for numbers 1 to 5 (each having its own 10 rows), which construct is best?

A. A single for loop
B. A do-while loop only
C. A nested for loop with an outer loop for numbers 1-5 and an inner loop for multipliers 1-10
D. No loop needed

Ans: C. A nested for loop with an outer loop for numbers 1-5 and an inner loop for multipliers 1-10

Explanation: This task requires iterating over 5 numbers and, for each, iterating over 10 multipliers — a classic nested for loop scenario.

Q89 **Output-based** *Loop Control Structure • Nested for Loop*

What is the output of: for(int i=1;i<=3;i++){ for(int j=1;j<=3;j++){ if(i==j) printf("%d ", i); } }

A. No output
B. 1 1 2 2 3 3
C. 1 2 3
D. 3 2 1

Ans: C. 1 2 3

Explanation: The condition i==j is true only on the diagonal cases (1,1), (2,2), (3,3), so it prints 1, 2, and 3.

Q90 **Conceptual** *Loop Control Structure • Nested for Loop*

What is the maximum practical limit on the depth of nesting for loops in C?

A. No fixed limit defined by the language itself, though excessive nesting reduces readability and performance

B. Exactly 2 levels only

C. Nesting is not allowed at all

D. Exactly 3 levels only

Ans: A. No fixed limit defined by the language itself, though excessive nesting reduces readability and performance

Explanation: C does not impose a strict language-level limit on nesting depth, though practical constraints such as readability, complexity, and performance discourage excessive nesting.

Q91 **Difficult** *Loop Control Structure • Nested for Loop*

What is the output of: int count=0; for(i=1;i<=3;i++){ for(j=1;j<=3;j++){ count++; } } printf("%d", count);

A. 3

B. 6

C. 12

D. 9

Ans: D. 9

Explanation: The inner loop executes 3 times for each of the 3 outer iterations, incrementing count each time, giving a total of 3 × 3 = 9.

CHAPTER 6 for Loop Variants

20 Questions

Q92 **Conceptual** *Loop Control Structure • for Loop Variants*

In C, is it mandatory to include all three expressions (initialization, condition, increment) inside a for loop header?

A. Only the condition is mandatory; others must be omitted

B. Yes, all three are mandatory and cannot be omitted

C. No, any or all of the three expressions can be omitted, but the semicolons must remain

D. Only initialization is mandatory

Ans: C. No, any or all of the three expressions can be omitted, but the semicolons must remain

Explanation: C allows any or all of the three expressions in a for loop to be omitted; however, the two semicolons separating them are always required.

Q93 **Special-Form** *Loop Control Structure • for Loop Variants*

What is the output/behavior of the code: int i=0; for(; i<5; i++) printf("%d ", i);

A. Infinite loop

B. Prints nothing

C. Works correctly, printing 0 1 2 3 4, since i was already initialized before the loop

D. Compilation error since initialization is missing

Ans: C. Works correctly, printing 0 1 2 3 4, since i was already initialized before the loop

Explanation: Since i is already initialized before the loop, omitting the initialization expression inside the for loop header is valid and the loop works normally.

Q94 **Special-Form** *Loop Control Structure • for Loop Variants*

What happens when the condition part of a for loop is omitted, e.g., for(i=0; ; i++)?

A. The loop executes exactly once

B. The loop becomes infinite unless explicitly broken using a statement like break

C. The loop never starts

D. Compilation error

Ans: B. The loop becomes infinite unless explicitly broken using a statement like break

Explanation: Omitting the condition makes it implicitly always true, resulting in an infinite loop unless a break statement or similar mechanism is used to exit.

Q95 **Special-Form** *Loop Control Structure • for Loop Variants*

What happens when the increment/decrement part of a for loop is omitted, e.g., for(i=0; i<5;)?

A. Compilation error

B. The loop stops immediately

C. The loop variable will not be automatically updated, potentially causing an infinite loop unless updated manually inside the body

D. The loop runs exactly 5 times regardless

Ans: C. The loop variable will not be automatically updated, potentially causing an infinite loop unless updated manually inside the body

Explanation: If the increment/decrement is omitted, the loop variable is not updated automatically; it must be updated manually within the loop body, otherwise the loop can run infinitely.

Q96 **Infinite-Loop** *Loop Control Structure • for Loop Variants*

Which of the following for loop headers creates an intentional infinite loop?

A. for(; ;)

B. for(int i=10; i>0; i--)

C. for(int i=0; i==10; i++)

D. for(int i=0; i<10; i++)

Ans: A. for(; ;)

Explanation: for(; ;) omits all three expressions, making the condition implicitly true forever, thus creating a classic infinite loop.

Q97 **Missing-Expression** *Loop Control Structure • for Loop Variants*

Which for loop variant has only the condition expression present, with both initialization and increment omitted?

A. for(i=0; ; i++)

B. for(i=0; i<5;)

C. for(; i<5;)

D. for(; ; i++)

Ans: C. for(; i<5;)

Explanation: for(; i<5;) has only the condition expression 'i<5' present; the initialization and increment slots are left empty.

Q98 **Multiple-Expressions** *Loop Control Structure • for Loop Variants*

Can a for loop header contain multiple statements in the increment section using the comma operator, such as for(i=0;i<5;i++,j--)?

A. No, this is invalid syntax

B. Yes, multiple expressions can be combined using commas in the increment section

C. Only if using pointers

D. Only in nested loops

Ans: B. Yes, multiple expressions can be combined using commas in the increment section

Explanation: C allows multiple expressions separated by commas in the increment/decrement section, enabling simultaneous updates to multiple variables.

Q99 **Special-Form** *Loop Control Structure • for Loop Variants*

What is the output of: for(int i=0; i<5; i++){} printf("%d", i);

A. 0

B. Compilation error, i is out of scope

C. 4

D. 5

Ans: B. Compilation error, i is out of scope

Explanation: Since i is declared within the for loop's own scope (for(int i=...)), it is not accessible outside the loop, causing a compilation error when printf tries to use it.

Q100 **Tricky** *Loop Control Structure • for Loop Variants*

Is for(; ;) { if(condition) break; } considered a valid technique to build a loop that runs until a condition is met?

A. No, it is invalid C syntax

B. Yes, it is a common technique using an infinite for loop combined with a break statement

C. It only works with while loops

D. It causes undefined behavior

Ans: B. Yes, it is a common technique using an infinite for loop combined with a break statement

Explanation: Using for(; ;) with an internal break statement is a valid and common C idiom for creating loops with custom, possibly complex, exit conditions.

Q101 **Difficult** *Loop Control Structure • for Loop Variants*

What is the output of: for(int i=0; i<5; i=i+2) printf("%d ", i);

A. 0 1 2 3 4

B. 0 2 4 6

C. 2 4

D. 0 2 4

Ans: D. 0 2 4

Explanation: Starting at 0 and increasing by 2 each time, the loop prints 0, 2, 4; the loop stops when i becomes 6, since 6<5 is false.

Q102 **Special-Form** *Loop Control Structure • for Loop Variants*

Which of the following is a valid for loop with a decrementing counter and missing initialization (already initialized before)?

A. for(--i; i>0; i--)

B. for(; i>0;)

C. int i=10; for(; i>0; i--)

D. for(i=10; i>0;)

Ans: C. int i=10; for(; i>0; i--)

Explanation: Since i is initialized before the loop, the header for(; i>0; i--) correctly omits only the initialization expression while retaining condition and decrement.

Q103 **Application** *Loop Control Structure • for Loop Variants*

Special forms of the for loop (with missing expressions) are useful in which scenario?

A. They serve no practical purpose

B. Only in recursive functions

C. When initialization or update logic needs to be handled outside the loop header or conditionally inside the body

D. Only for decorative purposes

Ans: C. When initialization or update logic needs to be handled outside the loop header or conditionally inside the body

Explanation: Special for loop forms are useful when initialization has already occurred earlier, or when update logic is conditional and better placed inside the loop body.

Q104 **Output-based** *Loop Control Structure • for Loop Variants*

What is the output of: int i; for(i=1; i<=5;){ printf("%d ", i); i++; }

A. No output

B. Infinite loop

C. 1 2 3 4 5

D. 1 2 3 4

Ans: C. 1 2 3 4 5

Explanation: Even though the increment is missing from the header, i is incremented inside the loop body, so the loop behaves normally and prints 1 through 5.

Q105 **Tricky** *Loop Control Structure • for Loop Variants*

What is a potential danger of omitting the increment/decrement expression in a for loop header without updating the variable elsewhere?

A. It automatically becomes a while loop
B. The program will not compile
C. The loop executes exactly zero times
D. It can lead to an infinite loop since the condition never becomes false

Ans: D. It can lead to an infinite loop since the condition never becomes false

Explanation: If the loop control variable is never updated anywhere, the condition will remain true indefinitely, resulting in an infinite loop.

Q106 **Conceptual** *Loop Control Structure • for Loop Variants*

In for(; ;), what is the implicit value of the omitted condition expression?

A. Zero
B. False
C. True
D. Undefined behavior

Ans: C. True

Explanation: When the condition expression is omitted in a for loop, C treats it as implicitly true, causing the loop to run indefinitely unless stopped by break.

Q107 **Code-Snippet** *Loop Control Structure • for Loop Variants*

What is the output of: for(int i=10; i>0; i-=3) printf("%d ", i);

A. 10 7 4 1
B. 1 4 7 10
C. 10 7 4 1 -2
D. 10 9 8

Ans: A. 10 7 4 1

Explanation: Starting from 10 and subtracting 3 each time gives 10, 7, 4, 1; the next value -2 fails the condition i>0, so the loop stops after printing 1.

Q108 **Assertion-Reason** *Loop Control Structure • for Loop Variants*

Assertion (A): for(; ;) always results in an infinite loop.
Reason (R): Omitting the condition expression makes it implicitly evaluate to true.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. A is true, R is false
C. Both A and R are true, but R does not explain A
D. A is false, R is true

Ans: A. Both A and R are true, and R correctly explains A

Explanation: The infinite behavior of for(; ;) directly results from the implicit true value of the omitted condition, so R correctly explains A.

Q109 **Difficult** *Loop Control Structure • for Loop Variants*

What is the output of: int i=0; for(; i<3;){ printf("%d ", i); i = i + 1; if(i==2) break; }

A. Infinite loop
B. 0 1 2
C. 0 1
D. 0

Ans: C. 0 1

Explanation: The loop prints 0, increments i to 1; loop continues, prints 1, increments i to 2, then break exits the loop since i==2, giving output '0 1'.

Q110 **Previous-Year-Pattern** *Loop Control Structure • for Loop Variants*

Which for loop variant is functionally equivalent to a while loop with the same condition?

A. for(init; condition;)
B. for(; ; increment)
C. for(; condition;)
D. for(init; ;)

Ans: C. for(; condition;)

Explanation: for(; condition;) with only the condition present behaves exactly like a while(condition) loop, since initialization and increment are handled elsewhere or omitted.

Q111 **Scenario** *Loop Control Structure • for Loop Variants*

A programmer wants a for loop where the loop variable is decremented by a variable step size determined at runtime, e.g., step. Which increment expression is valid?

A. for(i=0;i<10;+step)
B. for(i=0;i<10;i+=step)
C. for(i=0;i<10;i--step)
D. for(i=0;i<10;i++step)

Ans: B. for(i=0;i<10;i+=step)

Explanation: i+=step is valid C syntax and correctly increments i by the runtime-determined value stored in the variable step.

CHAPTER 7 The while Loop Statement

20 Questions

Q112 **Syntax** *Loop Control Structure • The while Loop Statement*

What is the correct syntax for a while loop in C?

A. while { condition } statements;

B. while(condition) { statements; }

C. while condition { statements; }

D. do { statements; } while(condition)

Ans: B. while(condition) { statements; }

Explanation: The correct while loop syntax is while(condition) { statements; }, where the condition is checked before each execution of the body.

Q113 **Working-Mechanism** *Loop Control Structure • The while Loop Statement*

How does a while loop determine whether to execute its body?

A. It executes the body a fixed number of times

B. It always executes the body once before checking the condition

C. It checks the condition first; if true, it executes the body, then checks again

D. It never checks any condition

Ans: C. It checks the condition first; if true, it executes the body, then checks again

Explanation: A while loop checks its condition before each iteration; if the condition is true, the body executes, and the process repeats until the condition becomes false.

Q114 **Execution-Flow** *Loop Control Structure • The while Loop Statement*

What happens when the condition in a while loop becomes false?

A. An error is generated

B. Control exits the loop and moves to the statement following the loop

C. The loop restarts from the beginning

D. The program terminates entirely

Ans: B. Control exits the loop and moves to the statement following the loop

Explanation: When the while loop's condition evaluates to false, the loop terminates and control passes to the next statement after the loop.

Q115 **Characteristic** *Loop Control Structure • The while Loop Statement*

Which of the following best describes a key characteristic of the while loop?

A. It always executes exactly once
B. It cannot use a counter variable
C. It is a top-tested (pre-test) loop
D. It is a bottom-tested (post-test) loop

Ans: C. It is a top-tested (pre-test) loop

Explanation: The while loop checks its condition before executing the body, which classifies it as a top-tested (pre-test) loop.

Q116 **Application** *Loop Control Structure • The while Loop Statement*

The while loop is particularly useful when:

A. Only single statements need to run
B. The number of iterations depends on a condition that may not be predictable in advance
C. The exact number of iterations is known beforehand
D. No condition is ever needed

Ans: B. The number of iterations depends on a condition that may not be predictable in advance

Explanation: The while loop is ideal for situations where the number of repetitions depends on a dynamic condition rather than a fixed, pre-known count.

Q117 **Code-Snippet** *Loop Control Structure • The while Loop Statement*

What is the output of: int i=1; while(i<=5){ printf("%d ", i); i++; }

A. 1 2 3 4 5
B. 2 3 4 5
C. Infinite loop
D. 1 2 3 4

Ans: A. 1 2 3 4 5

Explanation: The loop starts at i=1 and continues while i<=5, incrementing i each time, printing 1 through 5.

Q118 **Output-based** *Loop Control Structure • The while Loop Statement*

What is the output of: int i=10; while(i>5){ printf("%d ", i); i--; }

A. 10 9 8 7 6 5
B. 10 9 8 7 6
C. 10 9 8 7
D. 5 6 7 8 9

Ans: B. 10 9 8 7 6

Explanation: The loop runs while i>5, printing 10 down to 6; when i becomes 5, the condition i>5 fails and the loop stops.

Q119 Tricky *Loop Control Structure • The while Loop Statement*

What happens if the condition of a while loop is false at the very first check?

A. The loop body never executes
B. A syntax error occurs
C. The loop body executes once and then stops
D. The loop runs infinitely

Ans: A. The loop body never executes

Explanation: Since while is a top-tested loop, if the condition is false initially, the body is skipped entirely and never executes.

Q120 Difficult *Loop Control Structure • The while Loop Statement*

What is the output of: int i=1; while(i<5){ i++; if(i==3) continue; printf("%d ", i); }

A. 2 4 5
B. 2 3 4
C. 4 5
D. 2 4

Ans: D. 2 4

Explanation: i increments each time before the print check; when i==3 the continue statement skips printing, so only i=2 and i=4 get printed before the loop ends at i=5.

Q121 Conceptual *Loop Control Structure • The while Loop Statement*

Which of the following is essential to include inside a while loop body to avoid an infinite loop (in most typical cases)?

A. A statement that eventually makes the condition false
B. A break statement always
C. A print statement
D. A nested loop

Ans: A. A statement that eventually makes the condition false

Explanation: To ensure a while loop terminates, the body must contain logic (like incrementing/decrementing a variable) that eventually causes the condition to become false.

Q122 Scenario *Loop Control Structure • The while Loop Statement*

A program needs to keep asking the user for input until they enter a negative number, but it should not assume there will be at least one valid input before checking. Which loop is most natural?

A. for loop
B. No loop is needed
C. while loop
D. do-while loop

Ans: C. while loop

Explanation: Since the check should occur before any assumption about input, and the number of iterations is unknown, a while loop is the most natural fit here.

Q123 **Application** *Loop Control Structure • The while Loop Statement*

Which is a common real-world application of the while loop?

A. Compiling source code
B. Defining a function prototype
C. Repeating a task while a sensor value remains below a threshold
D. Declaring a constant

Ans: C. Repeating a task while a sensor value remains below a threshold

Explanation: The while loop is commonly used in scenarios like sensor monitoring, where the loop continues as long as a dynamic condition (like a threshold) holds.

Q124 **Fill-in-the-Blank** *Loop Control Structure • The while Loop Statement*

In C, the while loop is classified as a _____ loop because it checks its condition before executing the loop body.

A. Post-tested
B. Non-tested
C. Bottom-tested
D. Top-tested

Ans: D. Top-tested

Explanation: The while loop is a top-tested loop since the condition check always occurs before the loop body runs.

Q125 **Output-based** *Loop Control Structure • The while Loop Statement*

What is the output of: int i=0; while(i<3){ printf("Hi "); }

A. Hi (printed once)
B. Infinite loop, printing 'Hi' forever
C. Compilation error
D. Hi Hi Hi

Ans: B. Infinite loop, printing 'Hi' forever

Explanation: Since i is never updated inside the loop body, the condition i<3 always remains true, resulting in an infinite loop.

Q126 **Difficult** *Loop Control Structure • The while Loop Statement*

What is the output of: int i=5; while(i>0){ if(i==3) break; printf("%d ", i); i--; }

A. 5 4 3
B. 5 4 3 2 1
C. 5 4
D. 3 2 1

Ans: C. 5 4

Explanation: The loop prints 5 and 4; when i becomes 3, the break statement triggers before printing, so the output is '5 4'.

Q127 **Assertion-Reason** *Loop Control Structure • The while Loop Statement*

Assertion (A): The while loop can execute zero times.
Reason (R): The while loop checks its condition before executing its body.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. A is false, R is true
C. A is true, R is false
D. Both A and R are true, but R does not explain A

Ans: A. Both A and R are true, and R correctly explains A

Explanation: Because the while loop tests its condition first, a false condition at the start means zero executions occur, so R correctly explains A.

Q128 **Previous-Year-Pattern** *Loop Control Structure • The while Loop Statement*

Which of the following statements about the while loop is TRUE?

A. It always executes the body at least once
B. It cannot be combined with break or continue
C. It requires only a condition inside its parentheses; initialization and update are handled separately
D. It requires an initialization inside its parentheses

Ans: C. It requires only a condition inside its parentheses; initialization and update are handled separately

Explanation: The while loop's parentheses contain only the condition; any initialization and increment/decrement must be written separately, outside or inside the loop body.

Q129 **Tricky** *Loop Control Structure • The while Loop Statement*

Which of these is a valid way to create an intentional infinite while loop in C?

A. while(1)
B. while(0)
C. while(false)
D. while()

Ans: A. while(1)

Explanation: while(1) uses a condition that is always true (non-zero), thus creating an intentional infinite loop, typically exited using break.

Q130 **Code-Snippet** *Loop Control Structure • The while Loop Statement*

What is the output of: int i=2; while(i<=10){ printf("%d ", i); i = i * 2; }

A. 2 4 6 8 10
B. 2 4 8 16
C. 2 4 8
D. 2

Ans: C. 2 4 8

Explanation: Starting at i=2 and doubling each time: 2, 4, 8; the next value 16 fails the condition i<=10, so the loop stops after printing 2, 4, 8.

Q131 **Conceptual** *Loop Control Structure • The while Loop Statement*

In a while loop, where is the update (increment/decrement) statement typically placed?

A. Inside the loop body, since the while header only contains the condition

B. Inside the condition parentheses only

C. Before the while keyword

D. It cannot be included anywhere

Ans: A. Inside the loop body, since the while header only contains the condition

Explanation: Since the while loop header contains only the condition, the update statement must be placed inside the loop body to eventually make the condition false.

CHAPTER 8 Simple and Nested while Loop

15 Questions

Q132 **Definition** *Loop Control Structure • Simple and Nested while Loop*

What is a simple while loop?

A. A while loop with no condition

B. A while loop that contains another while loop inside it

C. A while loop that never terminates

D. A single while loop used to repeat a block of statements based on one condition

Ans: D. A single while loop used to repeat a block of statements based on one condition

Explanation: A simple while loop is a single, standalone while loop that repeats its body based on one controlling condition.

Q133 **Definition** *Loop Control Structure • Simple and Nested while Loop*

What is a nested while loop?

A. A while loop with no body

B. A while loop combined with a for loop only

C. A while loop placed inside the body of another while loop

D. A while loop with two conditions joined by AND

Ans: C. A while loop placed inside the body of another while loop

Explanation: A nested while loop is formed when one while loop (inner) is placed inside the body of another while loop (outer).

Q134 **Structure** *Loop Control Structure • Simple and Nested while Loop*

In a nested while loop structure, which loop must have its condition become false first to exit the innermost repeating block?

A. The inner loop

B. Both simultaneously

C. Neither, they run independently

D. The outer loop

Ans: A. The inner loop

Explanation: The inner loop's condition must become false to exit the inner loop for a given outer iteration, before control returns to the outer loop.

Q135 **Execution-Sequence** *Loop Control Structure • Simple and Nested while Loop*

In a nested while loop, for each single iteration of the outer loop, how many times does the inner loop's full cycle execute?

A. The inner loop runs through its entire set of iterations once

B. Exactly once, partially

C. The inner loop runs indefinitely

D. The inner loop never executes

Ans: A. The inner loop runs through its entire set of iterations once

Explanation: For every outer loop iteration, the inner while loop completes its own full set of iterations (as determined by its own condition) before control returns outward.

Q136 **Application** *Loop Control Structure • Simple and Nested while Loop*

Nested while loops are useful for which of the following tasks?

A. Reading a single character

B. Printing two-dimensional patterns where the number of iterations is not fixed in advance

C. Terminating the program immediately

D. Declaring single variables

Ans: B. Printing two-dimensional patterns where the number of iterations is not fixed in advance

Explanation: Like nested for loops, nested while loops are useful for two-dimensional repetitive tasks, especially when iteration counts are condition-driven rather than fixed.

Q137 **Code-Snippet** *Loop Control Structure • Simple and Nested while Loop*

What is the output of: int i=1; while(i<=2){ int j=1; while(j<=2){ printf("%d%d ", i, j); j++; } i++; }

A. 12 22

B. 11 12 21 22

C. 11 12

D. 11 21 12 22

Ans: B. 11 12 21 22

Explanation: For i=1, the inner loop prints 11 and 12; for i=2, it prints 21 and 22, giving the sequence 11 12 21 22.

Q138 **Comparison** *Loop Control Structure • Simple and Nested while Loop*

What is a key similarity between simple and nested while loops?

A. Both use the same condition-checking mechanism (test before executing body)

B. Nested loops don't use any condition

C. Simple loops always run infinitely

D. They are unrelated concepts

Ans: A. Both use the same condition-checking mechanism (test before executing body)

Explanation: Both simple and nested while loops rely on the same top-tested condition-checking mechanism; the difference lies only in structural placement, not the checking method.

Q139 **Tricky** *Loop Control Structure • Simple and Nested while Loop*

In a nested while loop, if the inner loop's condition never becomes false, what is the likely result?

A. The inner loop is skipped entirely

B. The program enters an infinite loop stuck in the inner loop

C. The outer loop terminates immediately

D. A compilation error occurs

Ans: B. The program enters an infinite loop stuck in the inner loop

Explanation: If the inner while loop's condition never becomes false, the program gets stuck executing the inner loop indefinitely, never returning control to the outer loop.

Q140 **Difficult** *Loop Control Structure • Simple and Nested while Loop*

What is the output of: int i=1; while(i<=3){ int j=1; while(j<=i){ printf("*"); j++; } printf("\n"); i++; }

A. A square of 9 stars

B. A right triangle pattern of stars: *, **, ***

C. No output

D. A single row of 3 stars

Ans: B. A right triangle pattern of stars: *, **, *

Explanation: The inner loop runs j from 1 to i, so row 1 prints 1 star, row 2 prints 2 stars, and row 3 prints 3 stars, forming a triangle pattern, matching the nested for loop equivalent.

Q141 **Scenario** *Loop Control Structure • Simple and Nested while Loop*

A program must process a grid of unknown row and column counts determined by dynamic conditions rather than fixed numbers. Which loop structure fits best?

A. A single for loop

B. Nested while loops with conditions based on dynamic values

C. No loop is needed

D. A single if statement

Ans: B. Nested while loops with conditions based on dynamic values

Explanation: Since both row and column counts depend on dynamic, condition-based logic rather than fixed numbers, nested while loops are more suitable than nested for loops here.

Q142 **Execution-Sequence** *Loop Control Structure • Simple and Nested while Loop*

In a nested while loop, when does control finally exit the outer loop entirely?

A. As soon as the inner loop finishes once

B. When the inner loop's condition becomes true

C. Only when the outer loop's own condition becomes false

D. Never, nested loops cannot exit

Ans: C. Only when the outer loop's own condition becomes false

Explanation: The outer loop continues iterating (re-running the inner loop each time) until its own controlling condition evaluates to false.

Q143 **Conceptual** *Loop Control Structure • Simple and Nested while Loop*

Can a nested while loop have more than two levels of nesting (a while loop inside a while loop inside another while loop)?

A. Only if using arrays

B. Yes, C allows multiple levels of nested while loops, though excessive nesting can reduce readability

C. No, C limits nesting to exactly two levels

D. Only if using recursion instead

Ans: B. Yes, C allows multiple levels of nested while loops, though excessive nesting can reduce readability

Explanation: C does not restrict the depth of nested while loops; multiple levels are syntactically valid, though deep nesting can make code harder to read and maintain.

Q144 **Output-based** *Loop Control Structure • Simple and Nested while Loop*

What is the output of: int i=1; while(i<=2){ int j=1; while(j<=3){ printf("%d ", j); j++; } i++; }

A. 1 2 3

B. 1 1 2 2 3 3

C. 2 3 4

D. 1 2 3 1 2 3

Ans: D. 1 2 3 1 2 3

Explanation: For each of the 2 outer iterations, the inner loop resets j to 1 and prints 1 2 3, so the total output is '1 2 3 1 2 3'.

Q145 **Difficult** *Loop Control Structure • Simple and Nested while Loop*

What is the output of: int i=1, count=0; while(i<=3){ int j=1; while(j<=2){ count++; j++; } i++; } printf("%d", count);

A. 2
B. 5
C. 3
D. 6

Ans: D. 6

Explanation: The inner loop runs 2 times for each of the 3 outer iterations, incrementing count each time, resulting in a total of 3 × 2 = 6.

Q146 **Previous-Year-Pattern** *Loop Control Structure • Simple and Nested while Loop*

Which of these correctly distinguishes a nested while loop from a nested for loop in terms of structure?

A. Nested for loops cannot have variables
B. Nested while loops require initialization and update to be handled explicitly inside the loop body/before the loop, unlike the compact for loop header
C. There is no difference; they behave identically in every respect
D. Nested while loops cannot use break or continue

Ans: B. Nested while loops require initialization and update to be handled explicitly inside the loop body/before the loop, unlike the compact for loop header

Explanation: Unlike the for loop's compact header, while loops require the programmer to manage initialization and update expressions separately, which is a key structural distinction in nested contexts.

CHAPTER 9 Increment and Decrement Operators

20 Questions

Q147 Basic *Loop Control Structure • Increment and Decrement Operators*

Which operator is used to increase the value of a variable by 1 in C?

A. +=
B. ++
C. ->
D. **

Ans: B. ++

Explanation: The ++ operator is the increment operator in C, used to increase a variable's value by 1.

Q148 Basic *Loop Control Structure • Increment and Decrement Operators*

Which operator is used to decrease the value of a variable by 1 in C?

A. <-
B. //
C. %=
D. --

Ans: D. --

Explanation: The -- operator is the decrement operator in C, used to decrease a variable's value by 1.

Q149 Conceptual *Loop Control Structure • Increment and Decrement Operators*

How many forms does the increment operator (++) have based on its position relative to the variable?

A. One
B. Four
C. Three
D. Two: prefix and postfix

Ans: D. Two: prefix and postfix

Explanation: The increment operator has two forms: prefix (++x), where the increment happens before use, and postfix (x++), where it happens after use.

Q150 Definition *Loop Control Structure • Increment and Decrement Operators*

In prefix increment (++x), when is the value of x incremented relative to its use in an expression?

A. After the current expression is fully evaluated
B. It is never incremented
C. Only at the end of the program
D. Before the value is used in the expression

Ans: D. Before the value is used in the expression

Explanation: In prefix increment, the variable is incremented first, and the updated value is then used in the expression.

Q151 **Definition** *Loop Control Structure • Increment and Decrement Operators*

In postfix increment (x++), when is the value of x incremented relative to its use in an expression?

A. It is never incremented
B. Only in loops
C. Before the value is used in the expression
D. The original value is used first in the expression, and x is incremented afterward

Ans: D. The original value is used first in the expression, and x is incremented afterward

Explanation: In postfix increment, the current (original) value of x is used in the expression first, and only after that is x incremented.

Q152 **Code-Snippet** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int x=5; printf("%d", ++x);

A. Error
B. 5
C. 6
D. 4

Ans: C. 6

Explanation: Prefix increment (++x) increments x to 6 before it is used in printf, so the output is 6.

Q153 **Code-Snippet** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int x=5; printf("%d", x++);

A. Error
B. 5
C. 6
D. 4

Ans: B. 5

Explanation: Postfix increment (x++) uses the original value of x (5) in printf first, then increments x afterward, so the output is 5.

Q154 **Code-Snippet** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int x=5; printf("%d", --x);

A. Error
B. 6
C. 4
D. 5

Ans: C. 4

Explanation: Prefix decrement (--x) decreases x to 4 before it is used in printf, so the output is 4.

Q155 **Code-Snippet** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int x=5; printf("%d", x--);

A. Error
B. 4
C. 5
D. 6

Ans: C. 5

Explanation: Postfix decrement (x--) uses the original value of x (5) in printf first, then decrements x afterward, so the output is 5.

Q156 **Difficult** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int x=5, y; y = x++ + ++x; printf("%d %d", x, y);

A. 7 11
B. 7 10
C. 6 11
D. 7 12

Ans: D. 7 12

Explanation: x++ uses 5 (x becomes 6), then ++x increments x to 7 and uses 7; sum y = 5+7 = 12, and final x = 7, giving '7 12'.

Q157 **Application** *Loop Control Structure • Increment and Decrement Operators*

Increment and decrement operators are most commonly used in loops for which purpose?

A. Testing the loop's condition only
B. Declaring the loop variable's data type
C. Initializing arrays
D. Updating the loop control variable after each iteration

Ans: D. Updating the loop control variable after each iteration

Explanation: Increment/decrement operators are typically used in the update part of loops to move the loop control variable toward the termination condition.

Q158 **Code-Snippet** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: for(int i=0; i<3; i++) printf("%d ", i);
versus
for(int i=0; i<3; ++i) printf("%d ", i);
What can be said about their outputs?

A. They produce identical outputs, since the incremented value isn't used in an expression within the header itself

B. The second causes a compilation error

C. The first is invalid syntax

D. They produce different outputs

Ans: A. They produce identical outputs, since the incremented value isn't used in an expression within the header itself

Explanation: When the increment/decrement result is not used within a larger expression (as in a loop's update section alone), prefix and postfix forms behave identically in effect.

Q159 **Tricky** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int a=10; int b = a++ + a++; printf("%d", b);

A. Undefined/compiler-dependent in strict terms, but commonly evaluates using values 10 and 11

B. 22

C. 21

D. 20

Ans: A. Undefined/compiler-dependent in strict terms, but commonly evaluates using values 10 and 11

Explanation: Since a++ + a++ modifies 'a' more than once between sequence points, its behavior is undefined per the C standard, although many compilers commonly produce 21 using values 10 and 11 in practice.

Q160 **Conceptual** *Loop Control Structure • Increment and Decrement Operators*

Which of the following is TRUE about the increment operator '++' when applied to a variable?

A. It cannot be used inside loops

B. It always requires two operands

C. It can be applied to various data types including int, float, char, and pointers

D. It can only be applied to integer variables

Ans: C. It can be applied to various data types including int, float, char, and pointers

Explanation: The ++ operator works on multiple types including integers, floating-point numbers, characters, and pointers (with pointer arithmetic semantics).

Q161 **Output-based** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: char ch='A'; ch++; printf("%c", ch);

A. Error
B. B
C. A
D. a

Ans: B. B

Explanation: Incrementing a char variable increases its underlying ASCII value by 1; since 'A' has ASCII 65, incrementing gives 66, which corresponds to 'B'.

Q162 **Difficult** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int i=1; while(i++ <= 3) printf("%d ", i);

A. 2 3
B. 1 2 3
C. 1 2 3 4
D. 2 3 4

Ans: D. 2 3 4

Explanation: Each iteration checks the original value of i against 3 (postfix), then increments; so the condition passes for i=1,2,3, but by the time printf runs, i has already been incremented to 2,3,4 respectively.

Q163 **Application** *Loop Control Structure • Increment and Decrement Operators*

In the for loop header for(i=10; i>0; i--), what role does the -- operator serve?

A. Initialization
B. Declaring the variable
C. Decrementing the loop control variable toward termination
D. Condition testing

Ans: C. Decrementing the loop control variable toward termination

Explanation: Here, -- serves as the update expression, decreasing i by 1 each iteration, moving it toward the condition i>0 becoming false.

Q164 **Previous-Year-Pattern** *Loop Control Structure • Increment and Decrement Operators*

Which statement correctly differentiates prefix and postfix increment when used as a standalone statement (not part of a larger expression), such as i++; or ++i; on its own line?

A. They produce the same final value of i, since the increment effect is identical regardless of position when used alone

B. Prefix is invalid in standalone form

C. They produce different final values of i

D. Postfix cannot be used in loops

Ans: A. They produce the same final value of i, since the increment effect is identical regardless of position when used alone

Explanation: When used as an isolated statement, both i++; and ++i; simply increase i by 1 with the same final result, since no surrounding expression uses the return value.

Q165 **Scenario** *Loop Control Structure • Increment and Decrement Operators*

A programmer wants to use the incremented value of a variable immediately within the same expression where it appears. Which form should typically be used?

A. Neither works for this purpose

B. Only decrement operators work this way

C. Postfix increment (x++)

D. Prefix increment (++x)

Ans: D. Prefix increment (++x)

Explanation: Prefix increment (++x) updates the variable first and then supplies the new (incremented) value for immediate use in the expression.

Q166 **Tricky** *Loop Control Structure • Increment and Decrement Operators*

What is the output of: int x=5; x = x++; printf("%d", x);

A. 5

B. 6

C. Undefined behavior, since x is both modified and assigned within the same expression without a sequence point

D. 7

Ans: C. Undefined behavior, since x is both modified and assigned within the same expression without a sequence point

Explanation: Modifying x via x++ and simultaneously assigning to x in the same statement without an intervening sequence point results in undefined behavior per the C standard, so no single guaranteed output is defined.

CHAPTER 10 Use of break

15 Questions

Q167 **Purpose** *Loop Control Structure • Use of break*

What is the primary purpose of the break statement in loops?

A. To skip the current iteration and continue with the next one

B. To pause program execution permanently

C. To restart the loop from the beginning

D. To immediately terminate the loop and transfer control to the statement after the loop

Ans: D. To immediately terminate the loop and transfer control to the statement after the loop

Explanation: The break statement's purpose is to immediately exit the enclosing loop (or switch), transferring control to the first statement after the loop.

Q168 **Working** *Loop Control Structure • Use of break*

When a break statement is executed inside a loop, what happens to the remaining iterations?

A. The loop restarts

B. Only the next iteration is skipped

C. They continue to execute normally

D. They are skipped, and the loop terminates immediately

Ans: D. They are skipped, and the loop terminates immediately

Explanation: Once break executes, all remaining iterations of that loop are skipped, and the loop terminates immediately.

Q169 **Termination** *Loop Control Structure • Use of break*

In a nested loop, if a break statement is executed inside the inner loop, which loop does it terminate?

A. Only the outer loop

B. Both inner and outer loops

C. Only the inner loop

D. Neither loop

Ans: C. Only the inner loop

Explanation: A break statement terminates only the innermost loop in which it is directly placed; outer loops continue executing normally.

Q170 **Code-Snippet** *Loop Control Structure • Use of break*

What is the output of: for(int i=1;i<=5;i++){ if(i==3) break; printf("%d ", i); }

A. 3 4 5
B. 1 2
C. 1 2 3
D. 1 2 3 4 5

Ans: B. 1 2

Explanation: The loop prints 1 and 2; when i becomes 3, break executes before printing, terminating the loop, so the output is '1 2'.

Q171 **Output-based** *Loop Control Structure • Use of break*

What is the output of: int i=1; while(i<=10){ if(i==4) break; printf("%d ", i); i++; }

A. 1 2 3 4 5 6 7 8 9 10
B. 1 2 3
C. 1 2 3 4
D. 4

Ans: B. 1 2 3

Explanation: The loop prints 1, 2, 3; when i reaches 4, break executes before the print statement, so the loop stops after printing '1 2 3'.

Q172 **Application** *Loop Control Structure • Use of break*

break is commonly used in which of the following scenarios?

A. Defining a function prototype
B. Searching for an element in a list and stopping as soon as it is found
C. Declaring a variable
D. Reading a header file

Ans: B. Searching for an element in a list and stopping as soon as it is found

Explanation: A classic use of break is in search algorithms, where the loop stops immediately once the desired element is found, avoiding unnecessary further iterations.

Q173 **Difficult** *Loop Control Structure • Use of break*

What is the output of: for(int i=1;i<=3;i++){ for(int j=1;j<=3;j++){ if(j==2) break; printf("%d%d ", i, j); } }

A. 11 12 13
B. 11 21 31
C. 11 12 21 22 31 32
D. No output

Ans: B. 11 21 31

Explanation: The break exits only the inner loop when j==2, so for each outer i, only j=1 gets printed, giving 11, 21, 31.

Q174 **Conceptual** *Loop Control Structure • Use of break*

Can the break statement be used inside a switch statement as well as loops?

A. break can only be used inside if statements

B. break cannot be used in C at all

C. Yes, break can be used inside both loops and switch statements to exit them

D. No, break only works inside loops

Ans: C. Yes, break can be used inside both loops and switch statements to exit them

Explanation: The break statement is valid inside both loop constructs (for, while, do-while) and switch statements, exiting whichever one directly encloses it.

Q175 **Tricky** *Loop Control Structure • Use of break*

If a break statement is placed outside any loop or switch statement, what happens?

A. It has no effect at all

B. It creates an infinite loop

C. It causes a compilation error

D. It works normally and terminates the program

Ans: C. It causes a compilation error

Explanation: A break statement used outside a loop or switch construct is a syntax/compilation error, since there is nothing valid for it to terminate.

Q176 **Effect-on-Execution** *Loop Control Structure • Use of break*

What is the effect of using break inside a do-while loop when a certain condition is met?

A. The loop's condition is evaluated one more time before terminating

B. break has no effect in do-while loops

C. The loop restarts

D. The loop terminates immediately, skipping the condition check that would normally follow the body

Ans: D. The loop terminates immediately, skipping the condition check that would normally follow the body

Explanation: When break executes inside a do-while loop, the loop terminates immediately without evaluating the trailing while condition for that iteration.

Q177 **Scenario** *Loop Control Structure • Use of break*

A program searches through a list of 100 numbers to find the first negative number. Using break upon finding it primarily improves:

A. Efficiency, by avoiding unnecessary iterations after the target is found
B. Variable declaration
C. Memory allocation
D. Code readability only

Ans: A. Efficiency, by avoiding unnecessary iterations after the target is found

Explanation: Using break to exit as soon as the target is found improves efficiency by preventing the loop from continuing to check the remaining, now-irrelevant elements.

Q178 **Assertion-Reason** *Loop Control Structure • Use of break*

Assertion (A): break immediately terminates the nearest enclosing loop.
Reason (R): break transfers control unconditionally to the statement immediately following the loop.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. A is false, R is true
C. Both A and R are true, but R does not explain A
D. A is true, R is false

Ans: A. Both A and R are true, and R correctly explains A

Explanation: break does terminate the nearest enclosing loop precisely because it transfers control unconditionally past the loop, so R correctly explains A.

Q179 **Output-based** *Loop Control Structure • Use of break*

What is the output of: int i=0; do{ i++; if(i==3) break; printf("%d ", i); } while(i<5);

A. 1 2 3 4 5
B. 1 2 3
C. 3
D. 1 2

Ans: D. 1 2

Explanation: i becomes 1 (prints 1), then 2 (prints 2); on the third iteration i becomes 3 and break exits before printing, so output is '1 2'.

Q180 **Previous-Year-Pattern** *Loop Control Structure • Use of break*

Which of the following best describes the behavior of break in a for loop compared to letting the condition naturally become false?

A. break causes an abrupt, immediate exit regardless of the loop's normal condition, often before the condition would naturally fail

B. break only works after the condition naturally fails

C. They are always identical in every scenario

D. break restarts the initialization

Ans: A. break causes an abrupt, immediate exit regardless of the loop's normal condition, often before the condition would naturally fail

Explanation: break provides an abrupt exit point that can occur at any point in the loop body, independent of whether the loop's actual condition has become false yet.

Q181 **Difficult** *Loop Control Structure • Use of break*

What is the output of: int found=0; for(int i=1;i<=5;i++){ if(i==3){ found=1; break; } } printf("%d %d", i, found);

A. 3 1

B. Compilation error, i is out of scope

C. 3 0

D. 5 1

Ans: B. Compilation error, i is out of scope

Explanation: Since i is declared within the for loop header's own scope, it is not accessible in the printf statement outside the loop, causing a compilation error.

CHAPTER 11 Use of continue

15 Questions

Q182 **Purpose** *Loop Control Structure • Use of continue*

What is the primary purpose of the continue statement in loops?

A. To exit the switch statement

B. To skip the remaining statements in the current iteration and proceed to the next iteration

C. To restart the entire program

D. To terminate the loop immediately

Ans: B. To skip the remaining statements in the current iteration and proceed to the next iteration

Explanation: The continue statement skips the rest of the current iteration's statements and moves control to the next iteration of the loop.

Q183 **Execution-Flow** *Loop Control Structure • Use of continue*

In a for loop, what happens immediately after a continue statement executes?

A. The loop restarts from initialization

B. Control jumps to the increment/decrement expression, then the condition is re-evaluated

C. Nothing happens

D. The loop terminates entirely

Ans: B. Control jumps to the increment/decrement expression, then the condition is re-evaluated

Explanation: In a for loop, continue causes control to jump to the update (increment/decrement) expression, after which the condition is checked again for the next iteration.

Q184 **Execution-Flow** *Loop Control Structure • Use of continue*

In a while loop, what happens immediately after a continue statement executes?

A. The loop restarts from the first statement in the program

B. The loop terminates

C. It behaves like break

D. Control jumps directly to the condition check, skipping any remaining statements in the body

Ans: D. Control jumps directly to the condition check, skipping any remaining statements in the body

Explanation: In a while loop, continue skips any remaining body statements and jumps straight to the condition check for the next potential iteration.

Q185 **Code-Snippet** *Loop Control Structure • Use of continue*

What is the output of: for(int i=1;i<=5;i++){ if(i==3) continue; printf("%d ", i); }

A. 1 2 3 4 5
B. 1 2 4 5
C. 1 2 3
D. 3

Ans: B. 1 2 4 5

Explanation: When i==3, continue skips the printf for that iteration only, so the loop still processes i=4 and i=5, giving output '1 2 4 5'.

Q186 **Output-based** *Loop Control Structure • Use of continue*

What is the output of: int i=0; while(i<5){ i++; if(i==2) continue; printf("%d ", i); }

A. 2
B. 1 3 4 5
C. 1 2 3 4 5
D. 1 3 4 5 6

Ans: B. 1 3 4 5

Explanation: i increments each time before the check; when i==2, continue skips the print for that iteration, so the output is '1 3 4 5'.

Q187 **Conceptual** *Loop Control Structure • Use of continue*

How does continue differ from break in terms of loop control?

A. Neither affects loop execution
B. continue skips to the next iteration without terminating the loop; break terminates the loop entirely
C. continue terminates the loop; break skips to the next iteration
D. They behave identically in all cases

Ans: B. continue skips to the next iteration without terminating the loop; break terminates the loop entirely

Explanation: continue moves on to the next iteration while keeping the loop running, whereas break exits the loop completely, stopping all further iterations.

Q188 **Application** *Loop Control Structure • Use of continue*

continue is commonly used in which scenario?

A. Ending the program

B. Declaring a new variable

C. Skipping processing of invalid or unwanted data while still checking the rest of a list

D. Defining a new loop type

Ans: C. Skipping processing of invalid or unwanted data while still checking the rest of a list

Explanation: A typical use of continue is to skip specific unwanted elements (such as invalid data) within a loop while continuing to process the remaining elements.

Q189 **Difficult** *Loop Control Structure • Use of continue*

What is the output of: for(int i=1;i<=5;i++){ if(i%2==0) continue; printf("%d ", i); }

A. 1 3 5 7

B. 1 2 3 4 5

C. 2 4

D. 1 3 5

Ans: D. 1 3 5

Explanation: continue skips even values of i (2 and 4), so only the odd values 1, 3, and 5 are printed.

Q190 **Difficult** *Loop Control Structure • Use of continue*

What is the output of: int i=0; do{ i++; if(i==3) continue; printf("%d ", i); } while(i<5);

A. 1 2 3 4 5

B. 1 2 4 5

C. Infinite loop

D. 1 2 3

Ans: B. 1 2 4 5

Explanation: When i==3, continue jumps to the while condition check (still incrementing correctly on subsequent iterations), skipping only the print for i=3, giving output '1 2 4 5'.

Q191 **Tricky** *Loop Control Structure • Use of continue*

In a do-while loop, what does continue cause control to jump to?

A. The trailing while condition check, which then determines whether to loop again

B. Nothing, continue is invalid in do-while loops

C. The end of the program

D. The beginning of the loop body directly, skipping the condition test entirely

Ans: A. The trailing while condition check, which then determines whether to loop again

Explanation: In a do-while loop, continue causes control to jump to the condition test at the bottom of the loop, which then determines whether another iteration occurs.

Q192 **Scenario** *Loop Control Structure • Use of continue*

A program processes a list of numbers and must skip negative numbers while still summing all positive ones. Which statement helps achieve this within a loop?

A. break
B. goto
C. continue
D. return

Ans: C. continue

Explanation: continue is ideal here since it allows skipping the processing of negative numbers for that iteration while letting the loop continue examining the rest of the list.

Q193 **Assertion-Reason** *Loop Control Structure • Use of continue*

Assertion (A): continue does not terminate the loop.
Reason (R): continue only skips the remaining statements of the current iteration and proceeds to the next one.
Choose the correct option:

A. A is true, R is false
B. A is false, R is true
C. Both A and R are true, but R does not explain A
D. Both A and R are true, and R correctly explains A

Ans: D. Both A and R are true, and R correctly explains A

Explanation: Since continue merely skips ahead to the next iteration rather than stopping the loop, R correctly explains why A (the loop is not terminated) is true.

Q194 **Output-based** *Loop Control Structure • Use of continue*

What is the output of: for(int i=1;i<=3;i++){ for(int j=1;j<=3;j++){ if(j==2) continue; printf("%d%d ", i, j); } }

A. 11 21 31
B. No output
C. 11 12 13 21 22 23 31 32 33
D. 11 13 21 23 31 33

Ans: D. 11 13 21 23 31 33

Explanation: For each i, continue skips only j==2, so j=1 and j=3 are printed for each row, giving 11 13 21 23 31 33.

Q195 **Previous-Year-Pattern** *Loop Control Structure • Use of continue*

Which of the following statements about continue in a for loop is TRUE?

A. It does NOT skip the increment expression; the increment still executes before the next condition check

B. It skips the increment expression entirely

C. It causes a compilation error if used inside nested loops

D. It behaves exactly like break

Ans: A. It does NOT skip the increment expression; the increment still executes before the next condition check

Explanation: In a for loop, continue jumps to the increment/decrement expression (which still executes) before the condition is checked again, unlike in while loops where increment logic in the body would be skipped if placed after the continue.

Q196 **Tricky** *Loop Control Structure • Use of continue*

If continue is placed as the very last statement inside a loop body, what is its practical effect compared to not using it at all?

A. It changes the loop's behavior significantly

B. It causes the loop to terminate

C. It has essentially no practical effect, since there are no further statements to skip

D. It duplicates the current iteration

Ans: C. It has essentially no practical effect, since there are no further statements to skip

Explanation: If continue is the last statement in the loop body, there is nothing left to skip, so its presence has no practical effect on the loop's behavior.

CHAPTER 12 The do-while Loop

20 Questions

Q197 **Syntax** *Loop Control Structure • The do-while Loop*

What is the correct syntax for a do-while loop in C?

A. do while(condition) { statements; }

B. while(condition) { statements; } do;

C. do { statements; } while(condition);

D. do { statements; } until(condition);

Ans: C. do { statements; } while(condition);

Explanation: The correct syntax is: do { statements; } while(condition); — note the mandatory semicolon after the while condition.

Q198 **Execution-Flow** *Loop Control Structure • The do-while Loop*

In a do-while loop, in what order do the body and condition get evaluated?

A. Neither is evaluated

B. Both simultaneously

C. Condition first, then body

D. Body first, then condition

Ans: D. Body first, then condition

Explanation: The do-while loop executes the body first, and only afterward evaluates the condition to decide whether to repeat.

Q199 **Characteristic** *Loop Control Structure • The do-while Loop*

What is the defining characteristic of the do-while loop?

A. It never terminates

B. It always executes the loop body at least once, regardless of the condition

C. It cannot use variables

D. It is a top-tested loop

Ans: B. It always executes the loop body at least once, regardless of the condition

Explanation: Because the condition is checked after the body executes, the do-while loop is guaranteed to execute its body at least one time.

Q200 **Bottom-Tested-Nature** *Loop Control Structure • The do-while Loop*

Why is the do-while loop referred to as a bottom-tested loop?

A. Because its condition appears at the bottom (end) of the loop syntax and is evaluated after the body executes

B. Because it has no header section

C. Because it uses the bottom half of memory

D. Because it always runs backward

Ans: A. Because its condition appears at the bottom (end) of the loop syntax and is evaluated after the body executes

Explanation: The condition of a do-while loop is written and evaluated at the bottom of the loop, after the body has already executed, hence 'bottom-tested'.

Q201 **Syntax** *Loop Control Structure • The do-while Loop*

Which punctuation mark is mandatory at the end of a do-while loop's while(condition) part but not required after a regular while loop's closing brace?

A. Comma (,)

B. Colon (:)

C. Semicolon (;)

D. Question mark (?)

Ans: C. Semicolon (;)

Explanation: A do-while loop must end with a semicolon after the while(condition) part, which is a distinguishing syntactic requirement not present in a standard while loop.

Q202 **Code-Snippet** *Loop Control Structure • The do-while Loop*

What is the output of: int i=1; do{ printf("%d ", i); i++; } while(i<=5);

A. 1 2 3 4

B. 2 3 4 5

C. 5

D. 1 2 3 4 5

Ans: D. 1 2 3 4 5

Explanation: The loop prints 1 through 5, incrementing i each time, and stops when i becomes 6 since 6<=5 is false.

Q203 **Tricky** *Loop Control Structure • The do-while Loop*

What is the output of: int i=10; do{ printf("%d ", i); i++; } while(i<5);

A. Nothing is printed

B. Infinite loop

C. 10 (printed exactly once)

D. 10 11 12 ...

Ans: C. 10 (printed exactly once)

Explanation: Even though the condition i<5 is false from the start, the do-while loop guarantees at least one execution, so 10 is printed once before the loop exits.

Q204 **Application** *Loop Control Structure • The do-while Loop*

do-while loops are especially useful for which of the following applications?

A. Menu-driven programs where the menu must be shown at least once before checking the user's choice to exit

B. Situations where the loop body should never execute

C. Defining macros

D. Declaring global variables

Ans: A. Menu-driven programs where the menu must be shown at least once before checking the user's choice to exit

Explanation: Since the menu needs to be displayed before the program can know whether the user wants to exit, do-while's guaranteed first execution fits this scenario perfectly.

Q205 **Advantage** *Loop Control Structure • The do-while Loop*

What is a key advantage of the do-while loop over the while loop in input validation scenarios?

A. It guarantees the input is read and processed at least once before the condition is checked

B. It uses less memory

C. It never requires validation

D. It executes faster in all cases

Ans: A. It guarantees the input is read and processed at least once before the condition is checked

Explanation: do-while's structure guarantees the input-reading code runs at least once, which is essential for the initial read before any validation condition can be evaluated.

Q206 **Output-based** *Loop Control Structure • The do-while Loop*

What is the output of: int i=5; do{ i--; printf("%d ", i); } while(i>0);

A. 5 4 3 2 1

B. 5 4 3 2 1 0

C. 4 3 2 1 0

D. 4 3 2 1

Ans: C. 4 3 2 1 0

Explanation: i is decremented before printing each time: starts at 5, decremented to 4 (printed), continues down to 0 (printed), stopping when i>0 becomes false after printing 0.

Q207 **Difficult** *Loop Control Structure • The do-while Loop*

What is the output of: int i=1, sum=0; do{ sum += i; i++; } while(i<=4); printf("%d", sum);

A. 4

B. 6

C. 10

D. 15

Ans: C. 10

Explanation: The loop adds 1+2+3+4 = 10 before i becomes 5, at which point the condition i<=4 fails and the loop stops, so sum equals 10.

Q208 **Comparison** *Loop Control Structure • The do-while Loop*

Which of these correctly compares the while and do-while loops?

A. They are functionally identical in every case

B. while checks the condition before execution; do-while checks it after execution, guaranteeing at least one run

C. while always runs more times than do-while

D. do-while never checks any condition

Ans: B. while checks the condition before execution; do-while checks it after execution, guaranteeing at least one run

Explanation: The fundamental difference is the timing of the condition check: before the body (while) versus after the body (do-while), with do-while always guaranteeing at least one execution.

Q209 **Assertion-Reason** *Loop Control Structure • The do-while Loop*

Assertion (A): The do-while loop is ideal for menu-driven programs.
Reason (R): The do-while loop guarantees that its body executes at least once before checking the exit condition.
Choose the correct option:

A. Both A and R are true, but R does not explain A

B. A is false, R is true

C. Both A and R are true, and R correctly explains A

D. A is true, R is false

Ans: C. Both A and R are true, and R correctly explains A

Explanation: The suitability of do-while for menu-driven programs stems directly from its guarantee of at least one execution, correctly explaining the assertion.

Q210 **Fill-in-the-Blank** *Loop Control Structure • The do-while Loop*

The do-while loop is classified as a _____ loop.

A. Bottom-tested

B. Pre-tested

C. Non-tested

D. Top-tested

Ans: A. Bottom-tested

Explanation: The do-while loop is classified as a bottom-tested (post-tested) loop since its condition is evaluated after the body executes.

Q211 **Scenario** *Loop Control Structure • The do-while Loop*

A programmer wants to repeatedly prompt a user for a password until it is entered correctly, ensuring the prompt appears at least once. Which loop best fits this requirement?

A. for loop
B. No loop needed
C. do-while loop
D. while loop

Ans: C. do-while loop

Explanation: Since the prompt must appear at least once regardless of any prior condition, the do-while loop's guaranteed first execution is the best fit.

Q212 **Tricky** *Loop Control Structure • The do-while Loop*

Forgetting the semicolon after while(condition) in a do-while loop results in:

A. A compilation error
B. An infinite loop only
C. A runtime warning that can be ignored
D. No issue at all, it's optional

Ans: A. A compilation error

Explanation: The semicolon after the while(condition) in a do-while loop is mandatory in C; omitting it causes a compilation error.

Q213 **Difficult** *Loop Control Structure • The do-while Loop*

What is the output of: int i=1; do{ if(i==3){ i++; continue; } printf("%d ", i); i++; } while(i<=5);

A. 1 2 4
B. 1 2 3 5
C. 1 2 3 4 5
D. 1 2 4 5

Ans: D. 1 2 4 5

Explanation: When i==3, i is incremented to 4 and continue jumps to the condition check, skipping the print for that value, so the output is '1 2 4 5'.

Q214 **Conceptual** *Loop Control Structure • The do-while Loop*

Can a do-while loop be made to run infinitely, similar to for(; ;) or while(1)?

A. do-while always terminates after one iteration
B. Yes, using do{ ... } while(1); with a break statement to control exit
C. Only using goto instead
D. No, do-while cannot be infinite

Ans: B. Yes, using do{ ... } while(1); with a break statement to control exit

Explanation: do{ ... } while(1); creates an infinite do-while loop, commonly controlled using an internal break statement, just like while(1) or for(; ;).

Q215 **Previous-Year-Pattern** *Loop Control Structure • The do-while Loop*

Which of the following is a correct structural requirement unique to the do-while loop compared to for and while loops?

A. The keyword 'do' must precede the loop body, and 'while(condition);' must follow it, with a mandatory closing semicolon

B. It must always be nested

C. It requires curly braces even for a single statement

D. It cannot contain any statements

Ans: A. The keyword 'do' must precede the loop body, and 'while(condition);' must follow it, with a mandatory closing semicolon

Explanation: The do-while loop has a unique two-part structure: the 'do' keyword before the body and 'while(condition);' after it, ending with a required semicolon — a structure not shared by for or while loops.

Q216 **Output-based** *Loop Control Structure • The do-while Loop*

What is the output of: int i=0; do{ printf("%d ", i); i += 2; } while(i<6);

A. 0 2 4 6

B. 0 1 2 3 4 5

C. 0 2 4

D. 2 4 6

Ans: C. 0 2 4

Explanation: The loop prints 0, then increases i by 2 each time: prints 2 and 4; when i becomes 6, the condition i<6 fails, so the loop stops after printing 0, 2, 4.

CHAPTER 13 Comparison Among for, while, and do-while

20 Questions

Q217 **Difference** *Loop Control Structure • Comparison Among for, while, and do-while*

What is the fundamental difference between the for loop and the while loop in C?

A. for combines initialization, condition, and increment in one header; while requires them to be handled separately

B. while always executes at least once

C. for cannot use break or continue

D. for is bottom-tested, while is top-tested

Ans: A. for combines initialization, condition, and increment in one header; while requires them to be handled separately

Explanation: The key structural difference is that the for loop's header compactly combines all three control expressions, whereas the while loop's header contains only the condition.

Q218 **Difference** *Loop Control Structure • Comparison Among for, while, and do-while*

What is the fundamental difference between the while loop and the do-while loop?

A. while checks its condition before the body; do-while checks it after the body

B. do-while cannot use conditions

C. They have no structural difference

D. while always executes at least once; do-while may execute zero times

Ans: A. while checks its condition before the body; do-while checks it after the body

Explanation: while is a top-tested (pre-test) loop, checking the condition first, whereas do-while is a bottom-tested (post-test) loop, checking the condition after the body executes.

Q219 **Difference** *Loop Control Structure • Comparison Among for, while, and do-while*

What is the fundamental difference between the for loop and the do-while loop?

A. They are functionally identical

B. do-while cannot use loop variables

C. for cannot be nested; do-while can

D. for is top-tested and combines all control expressions in its header; do-while is bottom-tested and always executes at least once

Ans: D. for is top-tested and combines all control expressions in its header; do-while is bottom-tested and always executes at least once

Explanation: The for loop is top-tested with a compact header, while do-while is bottom-tested and guarantees at least one execution regardless of the header's condition.

Q220 **Similarity** *Loop Control Structure • Comparison Among for, while, and do-while*

Which of the following is a similarity shared by all three loop types (for, while, do-while) in C?

A. They all execute the body at least once
B. None of the above; they share no similarities
C. They all require exactly the same syntax
D. They all repeat a block of statements based on a controlling condition

Ans: D. They all repeat a block of statements based on a controlling condition

Explanation: Despite their syntactic and testing-order differences, all three loop types share the common purpose of repeating a block of statements based on a condition.

Q221 **Comparison** *Loop Control Structure • Comparison Among for, while, and do-while*

Which loops in C are classified as top-tested?

A. Only for
B. Only do-while
C. while and do-while
D. for and while

Ans: D. for and while

Explanation: Both the for loop and the while loop check their condition before executing the body, classifying both as top-tested loops.

Q222 **Selection-Criteria** *Loop Control Structure • Comparison Among for, while, and do-while*

When the exact number of iterations is known in advance, which loop is generally the best choice?

A. while loop
B. do-while loop
C. for loop
D. Any loop works equally well with no preference

Ans: C. for loop

Explanation: The for loop is generally preferred for a known iteration count because its header neatly encapsulates initialization, condition, and update together.

Q223 **Selection-Criteria** *Loop Control Structure • Comparison Among for, while, and do-while*

When the loop body must execute at least once regardless of the condition, which loop is generally the best choice?

A. for loop
B. None of these
C. while loop
D. do-while loop

Ans: D. do-while loop

Explanation: The do-while loop is specifically suited for situations requiring guaranteed execution of the body at least once before the condition is evaluated.

Q224 **Selection-Criteria** *Loop Control Structure • Comparison Among for, while, and do-while*

When the number of iterations is unknown and the body might not need to execute at all if the condition is initially false, which loop fits best?

A. do-while loop
B. for loop
C. goto
D. while loop

Ans: D. while loop

Explanation: The while loop is ideal here since it checks the condition first and can skip the body entirely if the condition is false from the start.

Q225 **Advantage** *Loop Control Structure • Comparison Among for, while, and do-while*

What is an advantage of the for loop over the while loop when iteration count is known?

A. It hides all variables from the program
B. It cannot be terminated with break
C. It groups initialization, condition, and update together, improving readability and reducing errors
D. It always runs faster at the machine-code level

Ans: C. It groups initialization, condition, and update together, improving readability and reducing errors

Explanation: Grouping all three control expressions together in one place makes the for loop easier to read and less error-prone for known-iteration-count scenarios.

Q226 **Limitation** *Loop Control Structure • Comparison Among for, while, and do-while*

What is a limitation of the do-while loop compared to the while loop?

A. It cannot use variables
B. It cannot use break or continue
C. Its body always executes at least once, which can be undesirable if the condition should have prevented any execution
D. It cannot be nested

Ans: C. Its body always executes at least once, which can be undesirable if the condition should have prevented any execution

Explanation: Because do-while always runs the body first, it may execute unwanted code once even when the condition was never intended to allow execution, unlike the while loop.

Q227 **Practical-Usage** *Loop Control Structure • Comparison Among for, while, and do-while*

Which loop type would be most practical for validating repeated user input where the input prompt must appear at least once?

A. do-while loop
B. No loop is suitable
C. while loop
D. for loop

Ans: A. do-while loop

Explanation: do-while is the most practical choice here since the input must be requested (and thus the loop body executed) at least once before validation can occur.

Q228 **Match-the-Following** *Loop Control Structure • Comparison Among for, while, and do-while*

Match each loop with its primary characteristic:
1. for A. Bottom-tested, executes at least once
2. while B. Top-tested, compact header with all three expressions
3. do-while C. Top-tested, condition-only header
Correct match:

A. 1-B, 2-C, 3-A
B. 1-B, 2-A, 3-C
C. 1-C, 2-A, 3-B
D. 1-A, 2-B, 3-C

Ans: A. 1-B, 2-C, 3-A

Explanation: for matches B (top-tested, compact header), while matches C (top-tested, condition-only), and do-while matches A (bottom-tested, executes at least once).

Q229 **Conceptual** *Loop Control Structure • Comparison Among for, while, and do-while*

Which statement correctly compares the flexibility of all three loops for achieving the same repetitive task?

A. Only for loops can perform repetitive tasks
B. Each loop type can only be used for a single unique kind of task
C. Any task achievable with one loop type can generally be rewritten using the other two with appropriate adjustments
D. while and do-while cannot replace a for loop under any circumstances

Ans: C. Any task achievable with one loop type can generally be rewritten using the other two with appropriate adjustments

Explanation: In C, for, while, and do-while are functionally interchangeable for most repetitive tasks, differing mainly in syntax and condition-testing timing rather than fundamental capability.

Q230 **Assertion-Reason** *Loop Control Structure • Comparison Among for, while, and do-while*

Assertion (A): The do-while loop is the only loop among the three guaranteed to execute its body at least once.
Reason (R): It is the only bottom-tested loop among for, while, and do-while.
Choose the correct option:

A. A is true, R is false
B. A is false, R is true
C. Both A and R are true, but R does not explain A
D. Both A and R are true, and R correctly explains A

Ans: D. Both A and R are true, and R correctly explains A

Explanation: Since do-while alone tests its condition after the body executes, it alone guarantees at least one execution, making R the correct explanation for A.

Q231 **Previous-Year-Pattern** *Loop Control Structure • Comparison Among for, while, and do-while*

Which of the following correctly ranks the loops from top-tested to bottom-tested based on when the condition check occurs relative to the body?

A. for and while are top-tested; do-while is bottom-tested
B. All three are top-tested
C. do-while is top-tested; for and while are bottom-tested
D. All three are bottom-tested

Ans: A. for and while are top-tested; do-while is bottom-tested

Explanation: for and while both check their condition before the body (top-tested), while do-while checks after the body (bottom-tested).

Q232 **Tricky** *Loop Control Structure • Comparison Among for, while, and do-while*

Which of the following is TRUE if the initial condition is false in all three loop types (for, while, do-while) with an otherwise identical body?

A. All three loops will execute the body exactly zero times
B. for and while will execute zero times, but do-while will execute the body exactly once
C. do-while will execute zero times, but for and while will execute once
D. All three loops will execute infinitely

Ans: B. for and while will execute zero times, but do-while will execute the body exactly once

Explanation: Because for and while are top-tested, a false initial condition means their bodies never execute; but do-while, being bottom-tested, still executes its body once before checking.

Q233 **Comparison** *Loop Control Structure • Comparison Among for, while, and do-while*

In terms of code compactness for a simple counting loop with a known range, which loop typically requires the fewest lines/expressions in its header?

A. while loop
B. They are all equally compact
C. for loop
D. do-while loop

Ans: C. for loop

Explanation: The for loop typically requires the fewest lines since it consolidates initialization, condition, and update into a single header line.

Q234 **Difficult** *Loop Control Structure • Comparison Among for, while, and do-while*

Consider these two loops:
(1) for(i=0;i<0;i++){ printf("A"); }
(2) do{ printf("A"); }while(i<0);
Assuming i starts at 0 in both cases, what is the difference in output?

A. Loop (1) prints nothing; loop (2) prints 'A' once
B. Loop (1) prints 'A' once; loop (2) prints nothing
C. Both print 'A' once
D. Both print nothing

Ans: A. Loop (1) prints nothing; loop (2) prints 'A' once

Explanation: Since the for loop is top-tested, i<0 is false immediately, so it never executes; the do-while loop, being bottom-tested, executes its body once before checking the same false condition.

Q235 **Application** *Loop Control Structure • Comparison Among for, while, and do-while*

For iterating a fixed number of times over an array of known size, which loop is conventionally used in most C programs?

A. for loop
B. goto-based loop
C. while loop
D. do-while loop

Ans: A. for loop

Explanation: Since array size is typically known in advance, the for loop is the conventional and most readable choice for fixed-count array iteration.

Q236 **Limitations** *Loop Control Structure • Comparison Among for, while, and do-while*

Which of the following is a shared limitation of all three loop types if the programmer forgets to update the loop control variable?

A. All three can potentially result in an infinite loop
B. The program fails to compile
C. Only the do-while loop can become infinite
D. Only the for loop can become infinite

Ans: A. All three can potentially result in an infinite loop

Explanation: Regardless of loop type, if the controlling variable is never updated toward making the condition false, any of the three loops can run infinitely.

CHAPTER 14 Mixed Review and Exam-Pattern MCQs

20 Questions

Q237 **Previous-Year-Pattern** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which loop is best described as executing 'zero or more times'?

A. Only do-while loop
B. for loop and while loop
C. do-while loop
D. Only for loop

Ans: B. for loop and while loop

Explanation: Since for and while are top-tested, they may execute zero times if the condition is initially false, or more times otherwise, hence 'zero or more times'.

Q238 **Previous-Year-Pattern** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which loop is best described as executing 'one or more times'?

A. None of these
B. do-while loop
C. for loop
D. while loop

Ans: B. do-while loop

Explanation: The do-while loop always executes its body at least once before checking the condition, making it 'one or more times'.

Q239 **Match-the-Following** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Match the concept to its correct description:
1. break A. Skips current iteration
2. continue B. Terminates the loop
3. do-while C. Bottom-tested loop
Correct match:

A. 1-B, 2-A, 3-C
B. 1-B, 2-C, 3-A
C. 1-C, 2-B, 3-A
D. 1-A, 2-B, 3-C

Ans: A. 1-B, 2-A, 3-C

Explanation: break terminates the loop (B), continue skips the current iteration (A), and do-while is the bottom-tested loop (C).

Q240 **Scenario** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

A programmer needs to print all numbers from 1 to 50 that are divisible by 3, skipping others but not stopping the loop early. Which statement is most appropriate to use?

A. continue
B. goto
C. break
D. return

Ans: A. continue

Explanation: continue is appropriate here since numbers not divisible by 3 should simply be skipped for that iteration, while the loop continues checking the rest.

Q241 **Assertion-Reason** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Assertion (A): A for loop can be rewritten as an equivalent while loop.
Reason (R): Both are top-tested loops that check the condition before executing the body.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. A is false, R is true
C. A is true, R is false
D. Both A and R are true, but R does not explain A

Ans: A. Both A and R are true, and R correctly explains A

Explanation: Since both loops are top-tested and check conditions similarly, a for loop can be restructured into an equivalent while loop, and R correctly explains this equivalence.

Q242 **Difficult** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

What is the output of: int i; for(i=1;i<=5;i++){ if(i==2) continue; if(i==4) break; printf("%d ", i); }

A. 1 3 4
B. 1 2 3 4 5
C. 1 3
D. 1 2 3

Ans: C. 1 3

Explanation: i=1 prints; i=2 is skipped by continue; i=3 prints; i=4 triggers break before printing, so the output is '1 3'.

Q243 **Tricky** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which of the following loop headers will result in a compilation error in standard C?

A. for(i=0, j=1; i<5; i++, j++)
B. for(;;)
C. for(int i=0; i<5; i++)
D. for(int i=0; i<5, i++)

Ans: D. for(int i=0; i<5, i++)

Explanation: Option D uses a comma instead of a semicolon between the condition and increment sections, which is invalid syntax and causes a compilation error.

Q244 **Output-based** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

What is the output of: int i=1, j=1; while(i<=3){ j=1; while(j<=3){ if(j==i) break; printf("%d%d ", i, j); j++; } i++; }

A. 21 31 32

B. 11 12 13

C. 11 21 31 32

D. No output

Ans: A. 21 31 32

Explanation: For i=1, inner loop breaks immediately at j==1 (no print); for i=2, prints 21 then breaks at j==2; for i=3, prints 31, 32 then breaks at j==3. Total output: '21 31 32'.

Q245 **Comparison** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

A student claims that while and do-while loops always produce identical output for the same condition and body. Is this claim generally true?

A. Yes, always true

B. Yes, but only for for loops

C. No, they can differ when the initial condition is false, since do-while still executes once

D. No, they always differ regardless of the condition

Ans: C. No, they can differ when the initial condition is false, since do-while still executes once

Explanation: The claim is false in general: if the initial condition is false, while skips the body entirely, but do-while still executes it once, producing different output.

Q246 **Application** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which control statement combination would be used to skip printing multiples of 5 but stop the loop entirely once a number greater than 40 is reached?

A. Only continue for both cases

B. continue for multiples of 5, break for numbers greater than 40

C. break for multiples of 5, continue for numbers greater than 40

D. Only break for both cases

Ans: B. continue for multiples of 5, break for numbers greater than 40

Explanation: continue appropriately skips only the specific unwanted values (multiples of 5), while break is used to fully terminate the loop once the numbers exceed 40.

Q247 **Fill-in-the-Blank** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

The process of updating a loop control variable using operators like ++ or -- is generally called _____ in loop terminology.

A. Increment/decrement (update)
B. Condition testing
C. Initialization
D. Declaration

Ans: A. Increment/decrement (update)

Explanation: Updating the loop control variable, typically with ++ or --, is referred to as the increment/decrement or update step of a loop.

Q248 **Difficult** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

What is the output of: int i=0, sum=0; do{ if(i%2==0){ i++; continue; } sum += i; i++; } while(i<6); printf("%d", sum);

A. 3
B. 15
C. 9
D. 6

Ans: C. 9

Explanation: Even i values (0,2,4) are skipped via continue after incrementing; odd values 1,3,5 are added to sum: 1+3+5 = 9.

Q249 **Tricky** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which of the following correctly identifies why an infinite loop might occur unintentionally in a for loop?

A. The loop body is empty
B. The increment expression correctly updates the loop variable toward the exit condition
C. The initialization is present
D. The condition expression never becomes false because the update step is missing or incorrect

Ans: D. The condition expression never becomes false because the update step is missing or incorrect

Explanation: An unintended infinite loop typically arises when the update logic fails to move the loop variable toward making the condition false.

Q250 **Previous-Year-Pattern** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which of the following is the correct order of evaluation in a for loop's very first iteration?

A. Initialization, Condition, Body
B. Increment, Condition, Body
C. Body, Condition, Increment
D. Condition, Initialization, Body

Ans: A. Initialization, Condition, Body

Explanation: The for loop first performs initialization (once), then checks the condition, and only then executes the body if the condition is true.

Q251 **Scenario** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

A programmer wants a loop that reads and validates passwords, re-prompting only while the password is incorrect, but the prompt must be shown at least once. Which combination best fits: loop type and control statement (if any needed for early exit)?

A. do-while loop, no break needed if condition alone suffices
B. No loop can handle this scenario
C. for loop with a break only
D. while loop with continue only

Ans: A. do-while loop, no break needed if condition alone suffices

Explanation: A do-while loop naturally fits since the prompt must appear at least once, and the condition (incorrect password) alone can control repetition without necessarily needing break.

Q252 **Difficult** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

What is the output of: int i; for(i=1;i<=3;i++){ int j; for(j=1;j<=3;j++){ if(i==j) continue; printf("%d%d ", i, j); } }

A. 11 22 33
B. 11 12 13 21 22 23 31 32 33
C. 12 13 21 23 31 32
D. No output

Ans: C. 12 13 21 23 31 32

Explanation: continue skips the diagonal cases where i==j; all other combinations (12,13,21,23,31,32) are printed.

Q253 **Conceptual** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which of the following best explains why the for loop is often described as more 'compact' than the while loop?

A. It does not require curly braces

B. It consolidates initialization, condition, and update expressions into a single header line, unlike while which requires them scattered separately

C. It cannot be nested, making it simpler

D. It uses fewer keywords overall

Ans: B. It consolidates initialization, condition, and update expressions into a single header line, unlike while which requires them scattered separately

Explanation: The for loop's single-line header combining all three control expressions is what gives it its reputation for compactness compared to the while loop.

Q254 **Output-based** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

What is the output of: int i=1; while(i<=3){ int j=1; do{ printf("%d%d ", i, j); j++; } while(j<=2); i++; }

A. 11 12 21 22 31 32

B. 11 21 31 12 22 32

C. 11 12 13

D. No output

Ans: A. 11 12 21 22 31 32

Explanation: For each outer i (1 to 3), the inner do-while runs j from 1 to 2, printing the pair each time, giving 11 12, 21 22, 31 32 in sequence.

Q255 **Assertion-Reason** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Assertion (A): continue and break behave identically when placed as the very last statement in a loop body with no code following them.
Reason (R): continue skips to the next iteration while break exits the loop, but if nothing follows either statement, their observable effect on the current iteration is the same.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. A is true, R is false

C. Both A and R are true, but R does not explain A

D. A is false, R is true

Ans: D. A is false, R is true

Explanation: A is false: even as the last statement, continue still allows the loop to proceed to further iterations, while break terminates the loop entirely, so their overall effect on the program differs, though R's specific claim about the current iteration alone is true.

Q256 **Comparison** *Loop Control Structure • Mixed Review and Exam-Pattern MCQs*

Which of the following best summarizes the overall relationship between for, while, and do-while loops in C?

A. They are entirely unrelated constructs with no shared purpose

B. They can never be used to solve the same problem

C. They all achieve repetition but differ in header structure and in when the condition is tested relative to the loop body

D. Only for and while are related; do-while is unrelated

Ans: C. They all achieve repetition but differ in header structure and in when the condition is tested relative to the loop body

Explanation: All three loops serve the shared purpose of repeating a block of code, differing mainly in their syntactic structure and the timing of their condition check.